PRIVACY-ENHANCED SYNTHETIC DATA GENERATION FOR AUTONOMOUS VEHICLE TRAINING

COMMON INTEGRATED SOLUTION REPORT

Team member	Student ID	Individual component
Priyadarshi S.A.S.D.	IT22309556	Privacy Integration
Yubitha S.	IT22541284	Dataset Generation
Apeksha M.K.S.R.	IT22110220	Utility Evaluation
Wijesinghe R.D.P.T.	IT22066916	Risk and Compliance

Report submitted in partial fulfilment of the requirements for the

Bachelor of Science (Honours) Degree in Information Technology

Specialising in Cyber Security

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. We declare that this common report is our own work. It contains no unacknowledged material previously submitted for a degree or diploma at any university or higher education institute. To the best of our knowledge, it contains no material previously published or written by another person. Any such material is acknowledged in the text. We grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this report. This right covers the whole report or any part of it, in print, electronic or other media. We retain the right to use the content in future work.

Academic integrity and AI disclosure. We have disclosed all use of generative AI and other automated tools, as the module policy requires. We remain responsible for the accuracy, originality, citations, code, data, analyses and conclusions in this report.

Student name	Student ID	Signature	Date
Priyadarshi S.A.S.D.	IT22309556		26/10/2026
Yubitha S.	IT22541284		26/10/2026
Apeksha M.K.S.R.	IT22110220		26/10/2026
Wijesinghe R.D.P.T.	IT22066916		26/10/2026

Supervisor certification. The above candidates carried out this research project under my supervision.

Role and name	Signature	Date
Supervisor: Prof. Anuradha Jayakody		26/10/26
Co-supervisor: Mr. Dinith Primal		26/10/2026

ABSTRACT

Perception models for autonomous vehicles are trained on road data that records real people. Synthetic data from driving simulators is not private by default. The simulator is tuned with real statistics, and trained models can leak their training records. Stored files can be read or changed, and privacy claims are rarely checked independently. This report presents a four-component framework that handles these needs in one pipeline. Privacy Integration adds Laplace noise to seven calibration statistics within a total budget of $\varepsilon = 1.8$. It also trains models with DP-SGD under a hard limit and attacks them. Dataset Generation captures CARLA frames and protects each file with SHA-256, AES-256 and role-based access control. Utility Evaluation screens the labels for poisoning and measures detection utility against real KITTI images. Risk and Compliance checks the evidence with separately written code, maps it to the GDPR and issues a signed certificate. We tested the framework on 10,000 frames from five towns. Membership inference fell from an AUC of 0.639 to between 0.504 and 0.515 on every private model. The cost was 0.16 to 0.21 in test AUC. On an overfitted model, $\varepsilon = 1$ removed 99.5% of the attacker's advantage. All encryption tests passed, and a changed byte was rejected. A detector trained on synthetic data kept 10.46% of the real-data mAP50, with an FID of 115.99. The independent checks found that an earlier calibration release spent $\varepsilon = 12.6$ while reporting 1.8. After the fix, every check passed. A later review found a fixed public seed in the calibration noise. It was corrected before the dataset was captured again. Privacy and verification can therefore be built into a synthetic data pipeline. The gap between synthetic and real data is still the main open problem.

Keywords: differential privacy, synthetic data, autonomous vehicles, privacy auditing, secure data pipeline

ACKNOWLEDGEMENTS

We thank our supervisor, **Prof. Anuradha Jayakody**, and our co-supervisor, **Mr. Dinith Primal**, for their guidance and feedback throughout this research. Their advice helped us narrow a broad idea down to four components that we could build and test on time.

We also thank the Sri Lanka Institute of Information Technology for its academic environment and resources. The academic staff of the Faculty of Computing commented on our work during the proposal and progress reviews. Their comments changed how we tested and reported our results.

Each of us built a separate component, but every component depended on the others. We thank each other for the cooperation, the code reviews and the cross-checks behind many decisions in this report. Finally, we thank the open-source communities behind the tools we used, including CARLA, PyTorch, Opacus, Ultralytics, PyCryptodome and ReportLab.

TABLE OF CONTENTS

1 INTRODUCTION AND PROJECT CONTEXT....1
1.1 Background and context....1
1.1.1 Differential privacy in a data pipeline....2
1.1.2 Membership inference and testing a privacy budget....3
1.1.3 Securing synthetic driving data....3
1.1.4 Measuring the utility of synthetic data....4
1.1.5 Independent auditing and legal requirements....5
1.2 Stakeholders and user needs....6
1.3 Alignment with the Sustainable Development Goals....7
1.4 Purpose and scope of the common report....7
2 OVERALL PROBLEM DEFINITION AND FOUR SUBPROBLEMS....9
2.1 Overall problem definition....9
2.2 Root causes and overall research gap....9
2.3 Subproblem and component mapping....11
2.3.1 SP1: Privacy at the entry points....12
2.3.2 SP2: Protection of generated files....12
2.3.3 SP3: Utility and label integrity....12
2.3.4 SP4: Independent verification and compliance....13
2.4 Overall aim and group objectives....13
2.5 Scope assumptions and constraints....14
3 OVERALL SOLUTION STRUCTURE....16
3.1 Integrated solution overview....16
3.2 High level architecture....17
3.3 Component 1 summary....19
3.4 Component 2 summary....20
3.5 Component 3 summary....22
3.6 Component 4 summary....23
3.7 End to end user and data flow....25
4 COMPONENT INTEGRATION AND COHESIVE OPERATION....27
4.1 Integration strategy....27

4.2 Interfaces data contracts and shared models....28
4.3 Shared nonfunctional requirements....31
4.4 Integrated workflow and exception handling....33
4.5 Team contribution and dependency matrix....34
5 INTEGRATED IMPLEMENTATION AND PROJECT EXECUTION....36
5.1 Common technology stack and infrastructure....36
5.1.1 Software structure of each component....37
5.2 Development and integration process....39
5.2.1 Problems found during integration....40
5.3 Deployment and operational configuration....41
5.4 Evidence of team execution....43
6 INTEGRATED TESTING EVALUATION AND VALIDATION....45
6.1 Evaluation framework and success criteria....45
6.2 Integration and end to end testing....47
6.3 Overall results....48
6.3.1 Privacy at calibration and training....49
6.3.2 Protection of generated files....52
6.3.3 Utility of the synthetic data....54
6.3.4 Independent verification....57
6.3.5 Combined view....60
6.4 Stakeholder and user validation....61
6.5 Achievement of common objectives....62
6.6 Integrated limitations and threats....63
7 COMMERCIALISATION SUSTAINABILITY AND RESPONSIBLE INNOVATION....65
7.1 Value proposition and beneficiaries....65
7.2 Market alternatives and differentiation....66
7.3 Business model and feasibility....67
7.4 Adoption intellectual property and compliance....68
7.5 Sustainable Development Goal contribution....69
8 CONCLUSION AND FUTURE DIRECTIONS....70
8.1 Overall conclusion....70

8.2 Principal integrated contribution.....70
8.2.1 Lessons learned.....71
8.3 Future directions.....71
REFERENCES.....73
APPENDICES.....77
Appendix A Integrated interface specification.....77
Appendix B Team contribution declaration.....78
Appendix C Academic integrity and AI use disclosure.....79

LIST OF FIGURES

Figure 3.1: Overall system overview of the R26-CS-010 framework.....16
Figure 3.2: Layered architecture of the framework.....17
Figure 3.3: Internal data flow of the Privacy Integration component.....20
Figure 3.4: Secure synthetic data generation pipeline.....21
Figure 3.5: Stages of the Utility Evaluation component.....23
Figure 3.6: Checks of the Risk and Compliance component, their inputs and their results on the final outputs.....24
Figure 3.7: Order of hand-offs between the components.....25
Figure 4.1: Layout of an AES-256-GCM encrypted dataset file.....30
Figure 5.1: Agile SCRUM cycle followed by the group.....39
Figure 5.2: Gantt chart of the project.....43
Figure 5.3: Work breakdown structure of the project.....43
Figure 6.1: Test AUC and attack AUC by privacy configuration (mean and one standard deviation over three seeds).....49
Figure 6.2: Attack AUC of the shadow-model and loss-threshold attacks.....51
Figure 6.3: Stress test: attack AUC of every seed with its 95% bootstrap interval; the black line marks the mean of each configuration.....52
Figure 6.4: Results of the Dataset Generation security audit.....53
Figure 6.5: Frames accepted and excluded by the poisoning gate in each town.....54
Figure 6.6: Class distribution and share of small objects in the synthetic dataset.....55
Figure 6.7: mAP50 and mAP50-95 of the four cross-domain experiments.....56
Figure 6.8: Per-class AP50 for training and testing within and across domains.....56
Figure 6.9: Target and final logged ε of the three private configurations.....58
Figure 6.10: Compliance certificate issued for the final outputs.....59
Figure 6.11: Composed calibration budget of the earlier and corrected releases.....61

LIST OF TABLES

Table 1.1: Stakeholders and their needs....6
Table 1.2: Stakeholder needs mapped to framework requirements and components...7
Table 2.1: Research gaps and the components that address them....10
Table 2.2: Subproblems and the components that address them....11
Table 2.3: Group objectives, components and evidence....13
Table 3.1: Shared services, external systems and users of the architecture....18
Table 3.2: Calibration queries, sensitivities and Laplace noise scale at ε ≈ 0.2571 per query (total ε = 1.8)....19
Table 3.3: Training configurations of Privacy Integration....20
Table 3.4: RBAC roles and permissions....22
Table 3.5: Cross-domain evaluation protocol....23
Table 3.6: Independent checks of Risk and Compliance....25
Table 4.1: Integration sequence and the check at each hand-off....27
Table 4.2: Main files exchanged between components....28
Table 4.3: Privatized calibration statistics of the two releases....29
Table 4.4: Shared nonfunctional requirements....31
Table 4.5: Shared threat model and controls....32
Table 4.6: Failure paths and responses....33
Table 4.7: Team contribution matrix....34
Table 4.8: Dependency matrix between components....35
Table 4.9: Shared outputs of the group....35
Table 5.1: Development and test environment of each component....36
Table 5.2: Main modules and commands of each component....37
Table 5.3: Problems found during development and integration....40
Table 5.4: Stages of the deployed pipeline....42
Table 6.1: Datasets and splits used in the evaluation....45
Table 6.2: Measures used in the evaluation....46
Table 6.3: Automated test suites of the framework....47
Table 6.4: Integration and system test results....48
Table 6.5: Summary of the main results of each component....48

Table 6.6: Privacy and utility by configuration (mean ± standard deviation over three seeds).....49
Table 6.7: Two-sided paired t-tests of each private configuration against the baseline (three seeds).....50
Table 6.8: Loss-threshold attack (mean ± standard deviation over three seeds; random guessing gives an AUC of 0.5 and a TPR equal to the FPR).....50
Table 6.9: Stress test on an overfitted model (mean over five seeds; one-sided paired t-tests against the non-private model).....51
Table 6.10: Summary of the Dataset Generation security audit (AES-256-GCM)....52
Table 6.11: Cross-domain evaluation results.....55
Table 6.12: FID interpretation scale.....57
Table 6.13: Results of the independent checks on the final outputs.....57
Table 6.14: GDPR mapping in the final compliance report.....58
Table 6.15: Cost of privacy at the data level and the model level.....60
Table 6.16: Comments of the second progress review panel and the team's responses 61
Table 6.17: Achievement of the group objectives.....62
Table 7.1: Value of the framework for each customer or user group.....65
Table 7.2: Existing alternatives compared with the framework.....66
Table 7.3: Main parts of the business model.....67
Table 8.1: Future directions mapped to components and group objectives.....72
Table A.1: Integrated interface specification.....77
Table B.1: Team contribution declaration.....78

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AES	Advanced Encryption Standard
AI	Artificial Intelligence
AP	Average Precision
AUC	Area Under the ROC Curve
AV	Autonomous Vehicle
CARLA	Car Learning to Act (open-source driving simulator)
CBC	Cipher Block Chaining
CPU	Central Processing Unit
CSV	Comma-Separated Values
DG	Dataset Generation (component)
DP	Differential Privacy
DP-SGD	Differentially Private Stochastic Gradient Descent
EQ	Evaluation Question
FID	Fréchet Inception Distance
FPR	False-Positive Rate
GCM	Galois/Counter Mode
GDPR	General Data Protection Regulation
GO	Group Objective
GPS	Global Positioning System
GPU	Graphics Processing Unit
HMAC	Hash-based Message Authentication Code
IEC	International Electrotechnical Commission
IoU	Intersection over Union
IP	Internet Protocol
ISO	International Organization for Standardization
IV	Initialization Vector
JSON	JavaScript Object Notation
KITTI	Karlsruhe Institute of Technology and Toyota Technological Institute (dataset)
LiDAR	Light Detection and Ranging
MAC	Message Authentication Code

Abbreviation	Meaning
mAP	Mean Average Precision
MIA	Membership Inference Attack
ML	Machine Learning
NIST	National Institute of Standards and Technology
PDF	Portable Document Format
PI	Privacy Integration (component)
PKCS	Public-Key Cryptography Standards
PNG	Portable Network Graphics
RBAC	Role-Based Access Control
RC	Risk and Compliance (component)
RDP	Rényi Differential Privacy
RGB	Red, Green, Blue
ROC	Receiver Operating Characteristic
SDG	Sustainable Development Goal
SGD	Stochastic Gradient Descent
SHA	Secure Hash Algorithm
SP	Subproblem
TPR	True-Positive Rate
UE	Utility Evaluation (component)
XOR	Exclusive OR
YAML	YAML Ain't Markup Language
YOLO	You Only Look Once (object detector)

1 INTRODUCTION AND PROJECT CONTEXT

1.1 Background and context

Autonomous vehicles use machine learning models to detect objects, follow lanes and plan their driving. These models learn from large amounts of labeled sensor data. Public datasets such as KITTI [1] and the Waymo Open Dataset [2] were recorded by sensor-equipped cars on public roads. The recordings were then labeled by hand, which is slow and costly.

Real road data also records people. Cameras capture faces and license plates. Repeated routes can reveal where a person lives or works. Under the General Data Protection Regulation (GDPR), recordings of identifiable people are personal data. Article 25 asks for data protection by design and by default. Article 32 asks for suitable technical measures to protect the data [3]. A company that trains perception models on real footage must therefore treat that footage as sensitive.

One common answer is to generate training data in a driving simulator such as CARLA [4]. The simulator labels every frame on its own, and no real person appears in its images. However, synthetic data solves only part of the privacy problem. The simulator is calibrated with real statistics, so real information still enters the pipeline. Models trained on the data can reveal which records were in their training set [5], [6]. Stadler et al. [7] showed that synthetic data does not protect its source without a formal mechanism. Generated files can also be read or changed once they sit on disk [8], [9]. Finally, developers rarely provide privacy evidence that a third party can check.

Our project, R26-CS-010, addresses these risks with a privacy-enhanced synthetic data generation framework. Four students built it. Each of us owned one component, and the components run as one pipeline. The rest of this section reviews the evidence behind each part of the problem. It follows the order in which data moves through our pipeline.

1.1.1 Differential privacy in a data pipeline

Differential privacy (DP) limits what the output of a computation reveals about any single record [10], [11]. Dwork et al. [12] defined it as a property of a randomized mechanism. Take two datasets D and D′ that differ in one record, and any set of outputs S. A mechanism M satisfies ε-differential privacy if:

$$\Pr[M(D) \in S] \leq e^{\varepsilon} \cdot \Pr[M(D') \in S]$$

The value ε is the privacy budget. A smaller ε gives stronger protection but needs more noise. Deep learning usually uses the relaxed (ε, δ) form. It allows the bound to fail with a small probability δ [11].

The Laplace mechanism answers a numerical query by adding Laplace noise with scale $b = \Delta f / \varepsilon$. The sensitivity Δf is the largest change that one record can cause in the answer [12]. Two properties matter for our pipeline. First, post-processing cannot weaken the guarantee, so clipping or rescaling a private answer is safe. Second, the budgets of mechanisms that read the same records add up under sequential composition [11]. Section 6.3 shows how this rule affected the calibration budget of our project.

For model training, Abadi et al. [13] proposed differentially private stochastic gradient descent (DP-SGD). The gradient of each sample is clipped to a maximum L2 norm C. Gaussian noise with standard deviation σC is added to the sum. The result is averaged over a batch of size L:

$$\tilde{g} = (1/L) \cdot (\Sigma_i \, \mathrm{clip}(g_i, C) + N(0, \sigma^2 C^2 I))$$

Clipping limits how far one record can move the weights, and the noise hides the remaining effect. Mironov [14] introduced Rényi differential privacy (RDP), which gives tighter accounting over many steps. Opacus [15] implements DP-SGD and an RDP accountant for PyTorch. It does not allow batch normalization, because that layer mixes samples in a batch. Private models therefore use group normalization instead [16].

Some researchers have applied DP to generative models. DP-GAN [17] adds noise to the discriminator gradients, and DP-CGAN [18] adds label conditioning. Both were tested on MNIST and medical records, not on driving data. Tian et al. [19] used

correlated differential privacy in federated learning for autonomous driving. Xie et al. [20] surveyed privacy risks in autonomous vehicles and the methods used against them. None of these works protects the statistics that configure a driving simulator. That is the first point where real data enters our pipeline.

1.1.2 Membership inference and testing a privacy budget

A privacy budget is a theoretical bound. Its practical meaning can only be seen by attacking the model. Shokri et al. [5] introduced the shadow-model membership inference attack. The attacker trains shadow models that copy the target. It records how they respond to their own training records and to other records. An attack classifier then learns to tell members from non-members. Yeom et al. [21] linked this leakage to overfitting and showed that the per-sample loss is a strong signal.

Average measures can hide risk. Carlini et al. [6] showed that an attack with a modest AUC can still expose a few records. AUC is the area under the ROC curve. They suggested reporting the true-positive rate (TPR) at a low false-positive rate (FPR) as well. Jayaraman and Evans [22] found that the leakage measured at a given ε varies widely between settings. Nasr et al. [23] showed that a worst-case attacker can reach the formal bound, while realistic attackers fall well short.

We took two lessons from these studies. A formal ε should always be tested with a real attack. The same attack should also succeed against a non-private model. Only then can a failed attack on a private model be trusted.

1.1.3 Securing synthetic driving data

CARLA [4] provides realistic towns, moving traffic, changing weather and several sensors. It can recreate rare or dangerous scenes without any real risk. SHIFT [24] showed that a large multi-task driving dataset can be generated entirely in CARLA. Real datasets such as nuScenes [25] remain the reference for realism. These works focus on realism and scale. They say little about how the generated files are stored and protected.

Security surveys identify this gap. Gao et al. [8] named data tampering, sensor spoofing and unauthorized access as major threats to autonomous driving. Xu et al. [9] found that data leakage and insecure storage are not handled well. Giannaros et al. [26]

noted that security controls are rarely built into the data pipeline itself. For a synthetic data pipeline, three threats matter most. Tampering changes images or labels so that a model learns incorrect patterns. Disclosure allows an unauthorized party to read or copy the files. Privilege misuse lets a user or process do more than its task needs.

Standard controls exist for each threat. SHA-256 maps any input to a 256-bit digest [27]. Changing one bit gives a completely different digest, so a stored digest works as a fingerprint. The Advanced Encryption Standard (AES) is the standard symmetric block cipher, with keys of up to 256 bits [28]. Cipher Block Chaining (CBC) mode gives confidentiality only. Galois/Counter Mode (GCM) adds an authentication tag that is checked during decryption [29]. With GCM, a changed ciphertext, nonce or tag makes decryption fail. A GCM nonce must never be reused with the same key. Role-based access control (RBAC) assigns permissions to roles, and each user acts through a role. It supports least privilege and deny by default.

1.1.4 Measuring the utility of synthetic data

Privacy is simple to achieve if usefulness is ignored. A dataset of pure noise is perfectly private, but it cannot train a useful model. A privacy-preserving pipeline must therefore measure how useful its output still is.

The You Only Look Once (YOLO) family treats object detection as a single regression problem [30]. YOLOv8 is a widely used version [31]. Detection quality is usually reported as mean Average Precision (mAP), as in the COCO benchmark [32]. For mAP50, a detection is correct if its intersection over union (IoU) with the true box is at least 50%. The mAP50-95 score averages over IoU thresholds from 50% to 95%, so it is stricter. When some classes are rare, per-class results are needed, because an overall score can hide them [33].

The Fréchet Inception Distance (FID) compares two image sets with features from a pre-trained InceptionV3 network [34]. A lower FID means the sets are more alike. FID does not depend on a task, so it adds a second view next to mAP. A fair utility study also needs real data that is independent of the calibration source. KITTI is a common benchmark for this purpose.

Labels from another party are a further point of attack. Wang et al. [35] surveyed such data poisoning attacks. Shifted boxes can teach a detector to place objects in the wrong position. Injected labels can make it see objects that are not there. Labels from another component should therefore be checked before training starts.

1.1.5 Independent auditing and legal requirements

Several studies show that synthetic data and trained models can still leak information. Hayes et al. [36] showed that generative models are open to membership inference. van Breugel et al. [37] proposed DOMIAS, an attack on synthetic data that targets local overfitting. Murakonda and Shokri [38] built ML Privacy Meter so that organizations can measure the privacy risk of their models.

Re-identification is a separate risk. Sweeney [39] showed that a few quasi-identifiers can make a person unique and proposed k-anonymity as a defense. Pyrgelis et al. [40] showed that even aggregate location data can reveal whether a person is present. For vehicles, Adeboye et al. [41] showed that camera data is open to location inference. Metadata is another weak point. JSON, CSV and log files can reveal paths, machine names or addresses, even when the images are encrypted. The ISO/IEC 27559 standard gives a de-identification framework for both content and metadata [42].

On the legal side, Article 5 of the GDPR sets out purpose limitation, data minimization and accountability. Article 17 covers the right to erasure, Article 25 covers privacy by design and Article 32 covers security of processing. Ganev [43] discussed when synthetic data can count as anonymous. He argued that both formal privacy and attack testing are needed. The NIST Privacy Framework organizes privacy risk management into five functions [44]. The NIST AI Risk Management Framework lists privacy and security as parts of trustworthy AI [45]. These frameworks ask for evidence, but they do not say how to produce it for a synthetic driving pipeline. Chapter 2 builds the common problem from this gap.

1.2 Stakeholders and user needs

The stakeholders were first identified in the project proposal. We then refined their needs through supervisor reviews and the progress review panels. Table 1.1 lists the primary and secondary stakeholders and their main needs.

Table 1.1: Stakeholders and their needs

Stakeholder	Type	Main need
Autonomous vehicle developers (machine learning engineers and data infrastructure leads)	Primary	Training data at scale with an auditable privacy record, without a long privacy review for every dataset
Road users (pedestrians, drivers and passengers)	Primary	Faces, plates and routes not exposed through training pipelines or trained models
Data protection officers and regulators	Secondary	Evidence of lawful processing that they can read and verify
Academic researchers	Secondary	Synthetic datasets that can be shared with a clear privacy and utility record

A typical user is a machine learning engineer at an autonomous vehicle company. This engineer wants realistic synthetic data. The engineer must also show the data protection officer that the data and models do not expose real people. Real data carries legal risk, and current synthetic data tools give no formal privacy proof. From these needs we derived four requirements. These are formal privacy where real data enters, protected files, measured utility and evidence that a third party can check.

The secondary stakeholders look at the same pipeline from a different angle. A data protection officer does not run the code. This person needs a short, readable record of what was protected, how it was tested and what its limits are. A researcher who shares a dataset needs assurance that the files cannot be read or changed in transit. The researcher also wants to know that the dataset can still train a model. Table 1.2 links each need to the part of the framework that answers it.

Table 1.2: Stakeholder needs mapped to framework requirements and components

Need	Framework requirement	Component
Real statistics must not expose road users	Formal privacy on the calibration statistics	PI
Trained models must not reveal their training records	DP-SGD with a hard budget limit, tested with attacks	PI
Stored and shared files must stay confidential and unchanged	Hashing, encryption and role-based access from capture	DG
Data must still train a useful detector	Utility measured against a real baseline, with checked labels	UE
Claims must be checkable by an independent party	Signed evidence, independent checks and a signed certificate	PI, RC
Evidence must link to legal duties	Mapping of each control to a GDPR article and a check	RC

We did not interview or survey any external stakeholders. The needs above come from the proposal analysis, and the supervisors and review panels checked them.

1.3 Alignment with the Sustainable Development Goals

The framework mainly supports two Sustainable Development Goals (SDGs). The first is SDG 9 (Industry, Innovation and Infrastructure). Its target 9.5 calls for stronger research and technology capability. Our framework gives developers a tested way to use synthetic data with a formal privacy record. The second is SDG 16 (Peace, Justice and Strong Institutions). Target 16.10 covers the protection of fundamental freedoms, and privacy is one of these. The framework keeps personal data out of training pipelines and gives regulators evidence they can verify.

The link to road safety under SDG 3 (target 3.6) and SDG 11 (target 11.2) is only indirect. Better access to privacy-protected training data might help improve perception models. However, we did not measure any effect on safety. Section 7.5 discusses this contribution and its limits.

1.4 Purpose and scope of the common report

This report covers the common problem and the integrated solution. It explains how the four components split the problem, how they connect and what the full pipeline

achieved. The four individual theses hold the detailed literature review, design options and experiments of each component. Here we summarize that work only where it helps to explain the complete solution.

The report has eight chapters. Chapter 2 defines the common problem, its root causes and the four subproblems. Chapter 3 describes the overall solution and summarizes each component. Chapter 4 explains how the components were joined, including the shared files, contracts and failure handling. Chapter 5 covers the shared technology, the development process and the evidence of team execution. Chapter 6 presents the integrated testing and results and maps them to the group objectives. Chapter 7 discusses commercialization, ethics and sustainability. Chapter 8 answers the common problem and lists the future work. The appendices give the interface specification, the contribution declaration and the AI use disclosure.

2 OVERALL PROBLEM DEFINITION AND FOUR SUBPROBLEMS

2.1 Overall problem definition

Autonomous vehicle developers want to train perception models on synthetic data. This removes the need to store footage of real people. However, synthetic data is not private by default. Its privacy also cannot currently be checked outside the team that produced it. Real statistics enter the simulator, trained models can leak membership, and stored files can be read or changed. Privacy controls also have a cost. The data may end up too weak to train a useful detector.

This gives the common problem of our project:

How can a synthetic data pipeline for autonomous vehicle training apply formal privacy where real information enters?

The same pipeline must also protect its files, measure its utility and give evidence that an independent party can verify.

Each part of this question has a specific meaning. "Formal privacy" means a guarantee with a stated budget. "Where real information enters" points to two places. These are the calibration statistics that tune the simulator and the training step that can memorize records. "Protect its files" covers the time after generation, when files sit on disk or move between machines. "Measure its utility" prevents the pipeline from gaining privacy at the cost of unusable data. "An independent party" means that the person who checks a claim did not write the code that makes it. If any part is missing, others cannot accept the pipeline as private.

2.2 Root causes and overall research gap

We traced the problem to four root causes.

- **Calibration is treated as configuration.** Simulators are tuned with statistics from real data, and these statistics are not protected [4]. Private generative models such as DP-GAN [17] were tested on small benchmarks, not on driving data.

- **Formal budgets are not tested.** Privacy budgets are often chosen for accuracy and rarely checked with attacks. Yet the leakage measured at a given ε can vary widely [22], [23].
- **Files are protected late, if at all.** Synthetic datasets are usually protected after generation. Security controls are rarely part of the data pipeline itself [8], [9], [26].
- **Claims cannot be checked.** Utility is rarely measured against real data, and labels from another party are trusted as they are [35]. Few privacy claims are tested by an independent party. Regulations ask for evidence but do not explain how to produce it for this kind of pipeline [43], [44].

Table 2.1 lists these limitations in more detail. Each row names the part of our framework that responds to it.

Table 2.1: Research gaps and the components that address them

Gap	Limitation of existing work	Approach in this project
Calibration privacy	Simulator statistics are computed from real data with no protection	Laplace mechanism on every statistic before it reaches the simulator (PI)
DP on driving data	DP training was mostly tested on benchmarks such as MNIST [13]	DP-SGD on synthetic CARLA frames with a hard budget limit (PI)
Formal bound and attack	Budgets are chosen for accuracy and rarely checked with attacks	Shadow-model and loss-threshold attacks with a non-private control, plus a stress test (PI)
Protection at generation	Synthetic data is protected after generation, if at all	SHA-256, AES-256 and RBAC inside the capture pipeline (DG)
Tested controls	Security controls are rarely checked with repeatable tests	Six-test security audit that runs without the simulator (DG)
Measured utility	Privacy work rarely measures whether the data can still train a detector	YOLOv8 mAP on synthetic and real data, plus FID (UE)
Label integrity	Data from another party is trusted without checks	Three-layer poisoning gate before training (UE)
Independent verification	Privacy claims are rarely checked by an independent party	Seven checks written apart from the code they test (RC)
Legal mapping	Controls are built without a clear link to legal duties	GDPR mapping in which each article is backed by a passing check (RC)

This leaves one overall research gap. No single framework protects real information at calibration and training and also protects the generated files. None also measures the remaining utility and has every claim checked independently. Each part depends on the others, so one component alone cannot close the gap.

2.3 Subproblem and component mapping

We split the common problem into four subproblems. They are separate, but they depend on each other. Table 2.2 maps each subproblem to its component and owner.

Table 2.2: Subproblems and the components that address them

Subproblem	Question	Component	Owner
SP1: Privacy at entry points	How can real information be protected where it enters the pipeline, and how can the protection be tested?	Privacy Integration (PI)	IT22309556
SP2: Protection of generated files	How can every generated file be kept confidential, tamper-evident and limited to authorized roles?	Dataset Generation (DG)	IT22541284
SP3: Utility and label integrity	Is the protected synthetic data still useful for training a detector, and are its labels untouched?	Utility Evaluation (UE)	IT22110220
SP4: Independent verification	Do the privacy controls work as claimed, and how does the evidence map to legal duties?	Risk and Compliance (RC)	IT22066916

All four subproblems are needed. Without SP1, real information would reach the simulator and the models with no protection. Without SP2, an attacker could still copy or change the data on disk. Without SP3, the pipeline might produce unusable data or train on poisoned labels. Without SP4, every privacy claim would depend on the word of its author. The links run in several directions. SP2 uses the privatized statistics from SP1, and SP1 and SP3 both use the protected files from SP2. SP1 trains on the clean labels from SP3, and SP4 checks the outputs of all three.

Each subproblem became the research question of one individual thesis. The sections below state these questions and explain why each one is needed.

2.3.1 SP1: Privacy at the entry points

Research question: Can differential privacy at calibration and training stop membership inference on synthetic driving data at a measurable cost?

Real information enters our pipeline at two points. The first is calibration, where real statistics configure the simulator. The second is training, where a model may memorize its records. A formal ε does not show how much an attacker can learn. SP1 therefore needs both a mechanism and a test. The Laplace mechanism should privatize the seven statistics before they leave the data owner. DP-SGD with an RDP accountant should keep training within a set budget. A membership inference attack should then work against a non-private model and fail against the private ones. The cost should appear as a measured drop in accuracy.

2.3.2 SP2: Protection of generated files

Research question: Can integrity, confidentiality and access control be built into a CARLA capture pipeline and shown to work?

Synthetic datasets are often stored as plain files and protected later, if at all. Until then, a file can be read, copied or changed without detection. A changed training set could harm a model in a safety-critical system. SP2 therefore asks for protection inside the capture step itself. Every captured file should be hashed, access-checked and encrypted. A repeatable audit should then show that decryption restores the exact bytes and that a changed file is detected.

2.3.3 SP3: Utility and label integrity

Research question: How much detection utility does privacy-preserving synthetic data keep compared with real data?

A dataset that cannot train a working detector has no practical use. SP3 measures the cost of privacy at the data level. It trains a YOLOv8 detector on synthetic data and tests it on synthetic and real images. A detector trained on real KITTI data gives the upper bound. FID measures how close the two image sets are. SP3 also guards training against tampered labels, because the labels come from another component.

2.3.4 SP4: Independent verification and compliance

Research question: Can an independent component verify the privacy claims of the pipeline and map them to GDPR duties?

A privacy claim made by its own developer is weak evidence. Article 5(2) of the GDPR makes the controller responsible for showing compliance. It does not say how to produce the proof. SP4 answers this with checks written by a member who did not build the other components. Their targets are the signed reports, budget logs, calibration noise log, dataset manifests, metadata and released files. Each check gives a clear verdict, and each verdict is linked to a GDPR article.

2.4 Overall aim and group objectives

The aim of the project is to design, build and evaluate a privacy-enhanced synthetic data generation framework. It should apply privacy, protect the files, measure utility and verify every claim across the whole pipeline. Table 2.3 lists the group objectives, the components involved and the evidence used to evaluate each one. The first three objectives are the measurable targets from the project proposal.

Table 2.3: Group objectives, components and evidence

No.	Group objective	Components	Evidence
GO1	Keep the cumulative training budget within $\varepsilon = 10$	PI, RC	Per-step budget logs and the independent budget audit
GO2	Keep membership inference success at or below 55% on private models	PI	Shadow-model and loss-threshold attacks with a non-private control, and a stress test
GO3	Keep perception accuracy within 15% of a real-data model	DG, UE	Cross-domain mAP against a KITTI baseline
GO4	Privatize calibration statistics before they reach the simulator, within $\varepsilon = 2$	PI, DG, RC	Privatized file, simulator profile, composition and reconstruction checks
GO5	Protect every generated file with hashing, encryption and access control	DG, UE, RC	Security audit and manifest checks after decryption
GO6	Check every label file for poisoning before training	UE	Poisoning gate with planted-defect tests

No.	Group objective	Components	Evidence
GO7	Verify the privacy claims independently and map them to the GDPR	RC	Seven independent checks, compliance report and signed certificate

GO1 to GO3 come from the proposal and have numeric targets. GO4 to GO7 were added as the component designs developed, because each subproblem needed its own measurable outcome. Where possible, the evidence comes from a component other than the one that did the work. GO1, for example, is produced by Privacy Integration but checked by Risk and Compliance. We set these objectives before the final experiments, so the results in Chapter 6 could be judged against fixed targets.

2.5 Scope assumptions and constraints

Scope. The framework covers the pipeline from aggregate calibration statistics to verified synthetic training data. This includes calibration, capture, protection, utility measurement, private training and auditing. Deploying a perception model in a vehicle is outside the scope. No generative model was trained on real records, because CARLA renders frames directly from its settings.

Assumptions. The calibration statistics were computed from the Waymo Open Dataset. We trusted the simulation host and the pipeline code. We did not fully trust storage or the people with access to it. KITTI was used only as an independent real-data baseline.

Constraints. We worked to a fixed academic timeline with limited hardware. Privacy training ran on a laptop GPU with 4 GB of memory, and detector training ran on a CPU. We used the Waymo Open Dataset and KITTI under their non-commercial licenses.

Exclusions. Some threats are outside the scope. They include attacks on the running simulation host, memory scraping, theft of an encryption key and side-channel attacks. The framework has no key management service and no user authentication for the access gateway. We did not run a formal user study with external users. Sections 6.6 and 8.3 return to these gaps.

Dependencies. Each component needed outputs from another one. Dataset Generation needed the privatized statistics. Utility Evaluation and Privacy Integration needed the encrypted dataset and its manifests. Privacy Integration also needed the clean labels in pipeline mode. Risk and Compliance needed the signed evidence, the noise log and the dataset. Chapter 4 explains how we managed these dependencies.

3 OVERALL SOLUTION STRUCTURE

3.1 Integrated solution overview

The framework is a single pipeline. It turns real driving statistics into synthetic training data that is protected, measured and verified. It starts on the data owner's machine, where the real aggregate statistics are computed. These statistics never leave that machine in raw form. At the other end is a release decision. It rests on verified privacy claims, a compliance report and utility figures. Figure 3.1 shows the flow.

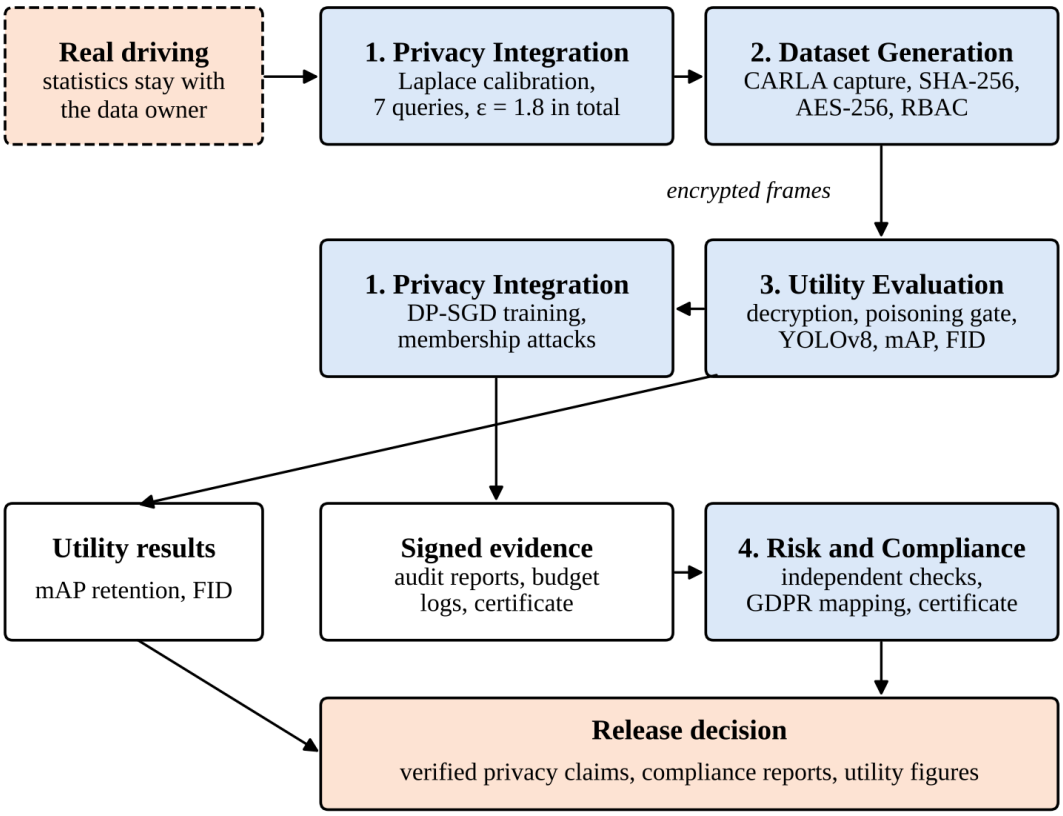


Figure 3.1: Overall system overview of the R26-CS-010 framework

In our framework, every privacy claim comes with a number, a test result and a signature. A party other than its author then checks it. A developer receives synthetic data together with a record of the work. The record shows where privacy was applied and how the files were protected. It also shows what privacy cost in utility and which checks passed.

We agreed on four design principles early, and each component follows them.

- **Privacy at the source.** Real information is privatized before it reaches the simulator or a model. No downstream stage sees the raw statistics.
- **Protection at capture.** Files are hashed and encrypted when they are created, so generated data does not stay on disk as readable plaintext.
- **Measure the cost.** Every privacy control is paired with a measure of its cost, at the model level and at the data level.
- **Independent checks.** The component that makes a claim signs it. A component that did not write the code then tests it.

3.2 High level architecture

The framework can also be seen as five layers, from the real source data up to the final assurance. Figure 3.2 shows these layers.

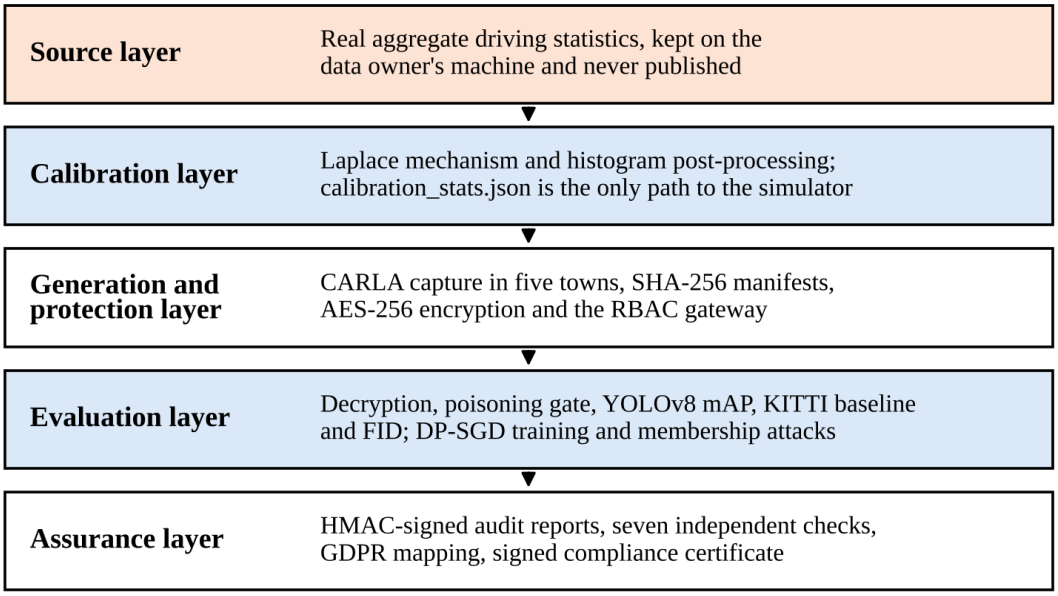


Figure 3.2: Layered architecture of the framework

- **Source layer.** Real aggregate driving statistics. They stay on the data owner's machine and are never published.
- **Calibration layer.** The Laplace mechanism in Privacy Integration turns the real statistics into privatized values in calibration_stats.json. This file is the only path from the source data to the simulator.
- **Generation and protection layer.** Dataset Generation captures frames in CARLA and protects them with SHA-256 manifests, AES-256 encryption and an RBAC gateway.

- **Evaluation layer.** Utility Evaluation decrypts the data, runs the poisoning gate and trains detectors. This layer also holds the DP-SGD training and attacks of Privacy Integration.
- **Assurance layer.** Privacy Integration writes HMAC-signed audit reports. Risk and Compliance runs seven independent checks, maps the results to the GDPR and signs a compliance certificate.

The architecture has three deployment boundaries. The data owner's machine holds the source statistics. The capture machine runs CARLA and the protection layer. The analysis machines hold the encrypted dataset and the keys needed for training, evaluation and auditing. Files cross these boundaries only when they are privatized, encrypted or signed. The users of the pipeline are the three roles of the RBAC gateway. A Simulation Engineer produces data, a Privacy Engineer checks it and an ML Researcher uses the processed datasets.

Besides the four components, the architecture has shared services, external systems and users. Table 3.1 lists them.

Table 3.1: Shared services, external systems and users of the architecture

Element	Type	Role in the framework
Shared repository	Shared service	Holds the code of all four components, the shared library, the contracts and the tests
JSON Schema contracts	Shared service	Define the format of every exchanged file; each file is checked when it is produced
Pipeline runner	Shared service	Runs the stages in order and stops at the first failed stage or gate
GitHub Actions	Shared service	Runs the tests of every component and an end-to-end pipeline test on each push
CARLA simulator	External system	Renders the synthetic towns, traffic and sensors from the privatized settings
KITTI dataset	External dataset	Gives an independent real-data baseline for the utility study
Simulation Engineer, Privacy Engineer, ML Researcher	Users	The three pipeline roles of the RBAC gateway
Reviewers and data protection officers	Users of the evidence	Read the signed certificates and the compliance report

3.3 Component 1 summary

Privacy Integration (owner: Priyadarshi S.A.S.D., IT22309556).

- **Subproblem:** SP1, privacy where real information enters.
- **Input:** Seven aggregate driving statistics, the encrypted dataset with its manifests and, in pipeline mode, the clean labels.
- **Core method:** The Laplace mechanism privatizes the seven statistics within a total budget of ε = 1.8. Opacus DP-SGD with an RDP accountant trains classifiers at ε = 1, 3 and 8, with a hard limit of 10 [13], [15]. A shadow-model attack [5] and a loss-threshold attack [21] test every model against a non-private control. A stress test repeats the attack on a deliberately overfitted model.
- **Output:** calibration_stats.json, a private calibration audit log, HMAC-signed audit reports, per-step budget logs and a signed privacy certificate.
- **Contribution:** Formal privacy at calibration and training, tested with real attacks and packaged as evidence that another party can verify.

Table 3.2: Calibration queries, sensitivities and Laplace noise scale at ε ≈ 0.2571 per query (total ε = 1.8)

Query	Type	Sensitivity Δf	Noise scale b
Vehicle density per km	Scalar	1.0	3.889
Pedestrian density per km	Scalar	1.0	3.889
Average vehicle speed (km/h)	Scalar	2.0	7.778
Average pedestrian speed (km/h)	Scalar	1.0	3.889
Intersections per km	Scalar	1.0	3.889
Weather distribution (4 bins)	Histogram	1.0	3.889 per bin
Time-of-day distribution (4 bins)	Histogram	1.0	3.889 per bin

Figure 3.3 shows how data moves inside the component. Calibration runs first, on the data owner's machine. Each statistic is one query, and Table 3.2 lists the sensitivities and noise scales. The total budget of 1.8 is split evenly, so each query runs at ε ≈ 0.2571. For the two distributions, noise is added to every bin. Negative bins are set to

zero and the bins are scaled to sum to one. This is post-processing, so it does not weaken the guarantee.

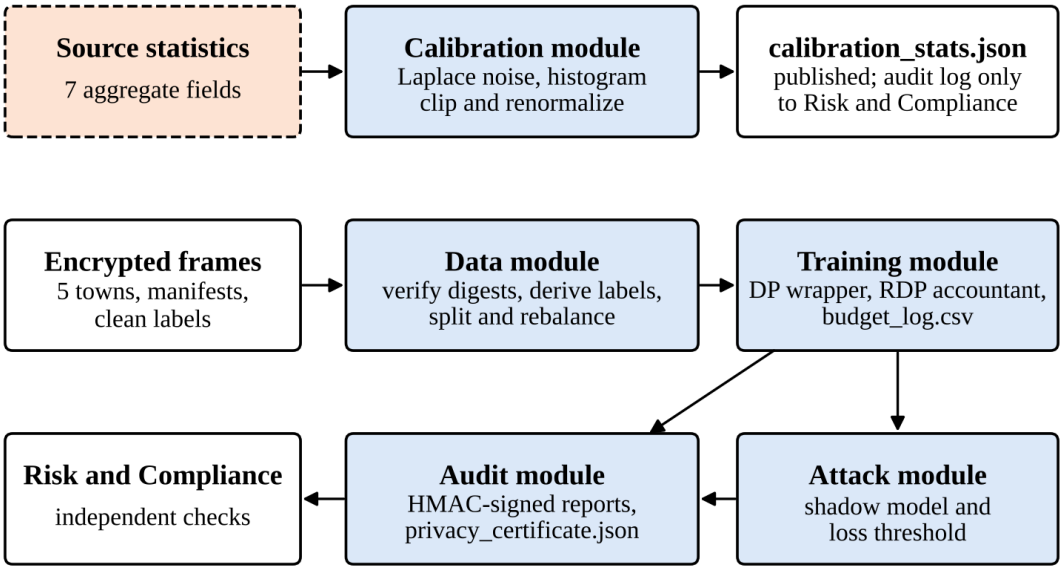


Figure 3.3: Internal data flow of the Privacy Integration component

The training side uses a small convolutional classifier with group normalization. Its task is binary. It decides whether a frame contains a pedestrian or cyclist large enough to see. Table 3.3 lists the four configurations, each trained with seeds 42, 43 and 44. The RDP accountant is read after every step, and the cumulative ε goes into a budget log. If ε reaches the hard limit of 10, training stops with an error.

Table 3.3: Training configurations of Privacy Integration

Configuration	Target ε	Mechanism	Seeds
No DP (baseline)	none	Adam, 15 epochs	42, 43, 44
Config A	1.0	DP-SGD, 10 epochs, C = 1.0, $\delta = 10^{-5}$, σ = 0.977	42, 43, 44
Config B	3.0	DP-SGD, 10 epochs, C = 1.0, $\delta = 10^{-5}$, σ = 0.665	42, 43, 44
Config C	8.0	DP-SGD, 10 epochs, C = 1.0, $\delta = 10^{-5}$, σ = 0.507	42, 43, 44

3.4 Component 2 summary

Dataset Generation (owner: Yubitha S., IT22541284).

- **Subproblem:** SP2, protection of the generated files.
- **Input:** The privatized calibration statistics and the CARLA simulator.
- **Core method:** Captures RGB, depth, semantic segmentation and class mask images in five towns. Each file is hashed with SHA-256 [27], checked against the RBAC policy and encrypted with AES-256 [28]. The delivered dataset uses CBC mode. The engine also supports GCM, which adds built-in tamper detection [29].
- **Output:** Encrypted town folders with SHA-256 manifests, per-frame metadata, YOLO labels and a security audit report.
- **Contribution:** Confidentiality, integrity and least-privilege access from the moment of capture.

Figure 3.4 shows the order of the protection steps. This order was chosen deliberately. The digest is taken from the plaintext as early as possible. The access check comes before any operation on the data. Encryption is the last step before data leaves the component. The security code lives in two classes. SecurityEngine hashes and encrypts, and RBACGateway checks permissions. Neither class depends on CARLA, so both can be tested without the simulator.

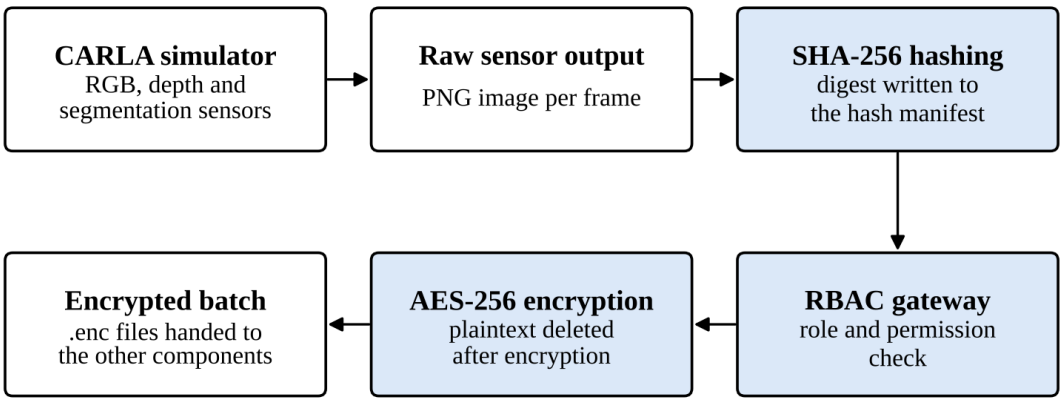


Figure 3.4: Secure synthetic data generation pipeline

The access gateway follows the real division of work in our group. Table 3.4 lists the three roles. A request is granted only if the role exists and the permission is in its list. Every other request is denied, and each decision is printed for review.

Table 3.4: RBAC roles and permissions

Role	Permitted operations	Purpose
Simulation_Engineer	create, verify, encrypt	Produces and protects raw simulation output
Privacy_Engineer	verify	Checks integrity before privacy processing
ML_Researcher	read_processed	Uses processed datasets only

3.5 Component 3 summary

Utility Evaluation (owner: Apeksha M.K.S.R., IT22110220).

- **Subproblem:** SP3, utility and label integrity.
- **Input:** The encrypted dataset with its manifests and labels, and the KITTI dataset as an independent real baseline.
- **Core method:** Decrypts the data with manifest checks and runs a three-layer poisoning gate. YOLOv8n detectors [31] are trained on synthetic and real data and tested across domains with mAP. FID [34] measures how close the two image sets are.
- **Output:** Clean labels, a poisoning audit, cross-domain mAP results, utility retention and the FID score.
- **Contribution:** A measured cost of privacy at the data level, and a gate that keeps tampered labels out of training.

Figure 3.5 shows the stages of the component. Decryption and the poisoning gate come first, so no detector is trained on data that failed a check. The gate has three layers. The structural layer checks every label line for valid fields and a known class. Each box must lie inside the frame and have a non-zero area. The batch layer tests each batch of 100 frames against the rest of its town with a chi-square test. A batch is rejected only when the result is both significant ($p < 0.0001$) and large (effect size above 0.40). The cross-town layer compares each town's class mix with the other four and raises a warning.

The utility measure uses four train and test combinations, listed in Table 3.5. Each model is tested on its own domain and on the other domain. The main figure is utility retention: synthetic-to-real performance as a share of real-to-real performance.

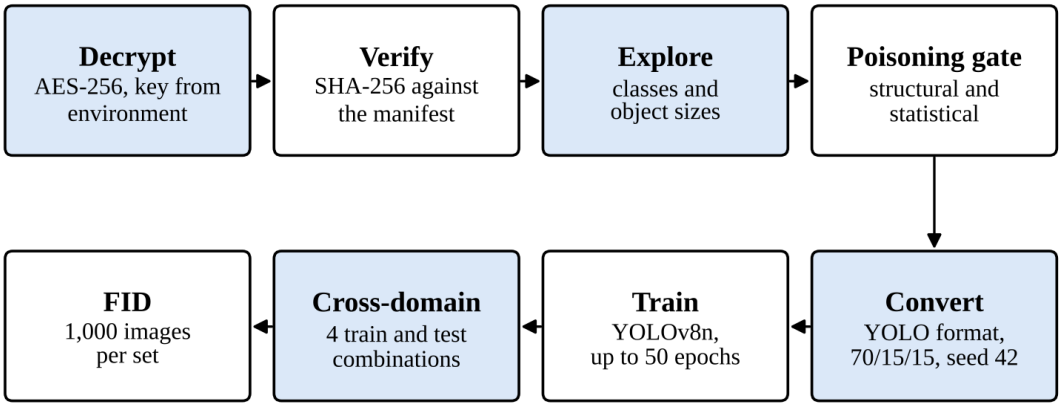


Figure 3.5: Stages of the Utility Evaluation component

Table 3.5: Cross-domain evaluation protocol

No.	Train domain	Test domain	Purpose
1	Synthetic	Synthetic	Can a model learn from this data at all?
2	Synthetic	Real (KITTI)	Transfer from synthetic to real, the main utility measure
3	Real (KITTI)	Real (KITTI)	Upper bound with real data
4	Real (KITTI)	Synthetic	Reverse check on the domain gap

3.6 Component 4 summary

Risk and Compliance (owner: Wijesinghe R.D.P.T., IT22066916).

- **Subproblem:** SP4, independent verification and legal mapping.
- **Input:** Signed reports, budget logs and calibration files from Privacy Integration. Manifests and metadata from Dataset Generation, and the poisoning audit from Utility Evaluation.
- **Core method:** Seven checks written apart from the code they test. They verify signatures and audit budgets. They check the calibration noise log and budget composition. They also re-hash the dataset, measure k-anonymity [39] and scan the released files for leakage.
- **Output:** One JSON result per check, a compliance report mapped to GDPR articles [3] and a signed PDF certificate.
- **Contribution:** Independent evidence that the controls of the other components work, with each control linked to a legal duty.

Figure 3.6 shows the checks, the evidence each one reads and its result on the final outputs. Each check returns a verdict. The compliance report then marks a GDPR provision as verified only when the check behind it has run and passed. The overall verdict is PASS when every check passed, FAIL when one failed and INCOMPLETE when one has not run.

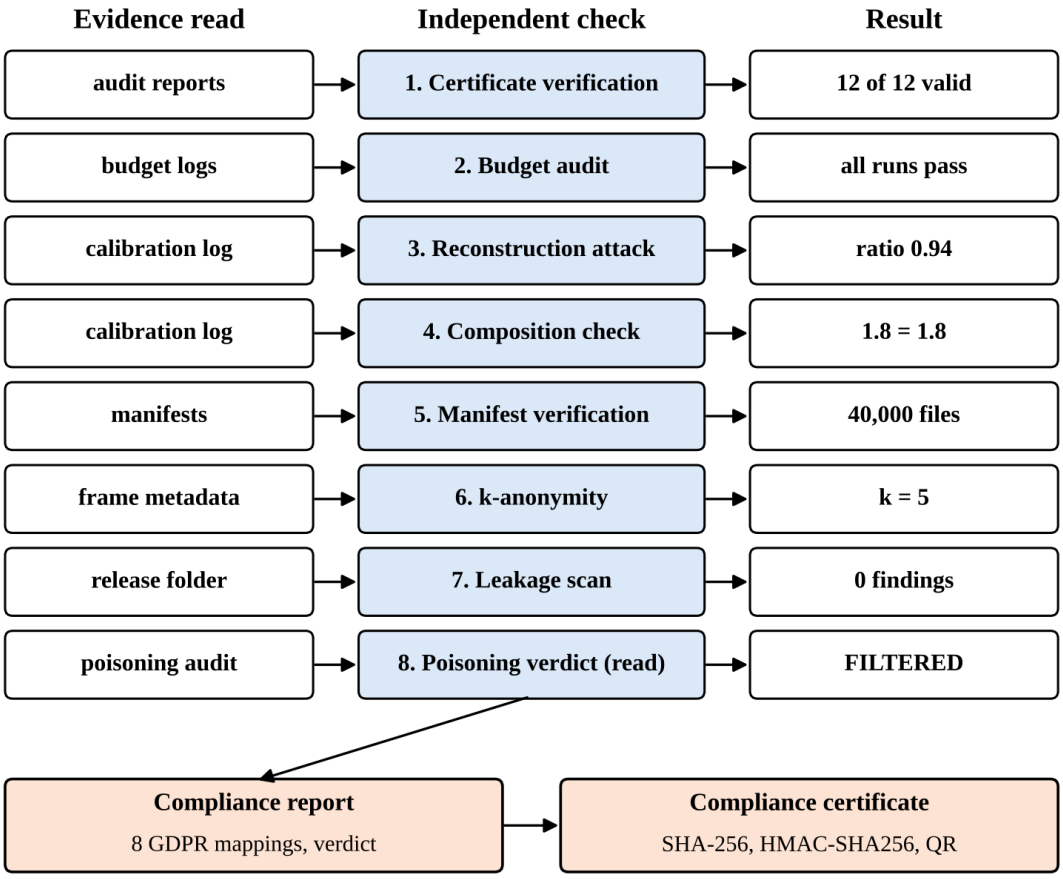


Figure 3.6: Checks of the Risk and Compliance component, their inputs and their results on the final outputs

Table 3.6 lists the checks and their pass rules. The calibration log check has two parts, a reconciliation and a reconstruction attack. No check reuses the signing, hashing or noise code of the component it tests. The compliance report combines six of these checks with the poisoning verdict of Utility Evaluation. The leakage scan runs last, on the release folder.

Table 3.6: Independent checks of Risk and Compliance

Check	Evidence it reads	Pass rule
Certificate verification	Signed audit reports from PI	Tag and digest match; a changed copy is rejected
Budget audit	Per-step budget logs and reported ε	Within 10^{-4} of the report, never decreasing, at or below 10
Calibration log: reconciliation	Source values, published values, noise log	Source value plus logged noise equals the published value
Calibration log: reconstruction attack	Published and source values	Observed over expected error between 0.3 and 3.0
Composition check	Noise log and published total	Sum of per-query ε at or below the claimed total
Manifest verification	Decrypted files and DG manifests	Every file matches; none missing
k-anonymity	Frame metadata	Smallest group of frames sharing all quasi-identifiers at least 5
Leakage scan	Released text files	No paths, addresses, coordinates or credentials; otherwise review

3.7 End to end user and data flow

A data item passes through the full solution in seven steps. Figure 3.7 shows the order of the hand-offs and the components at each step.

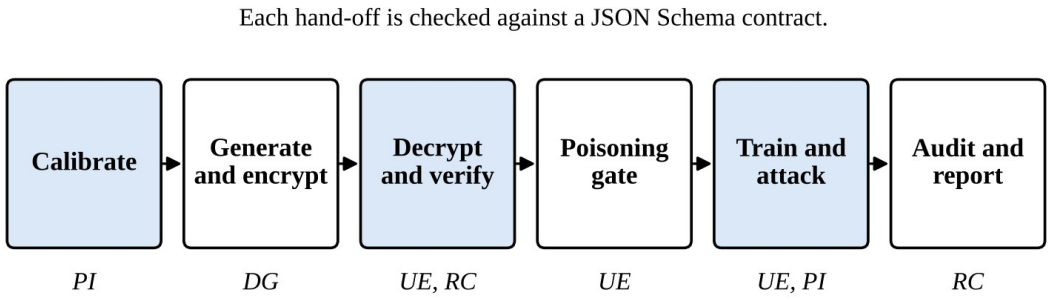


Figure 3.7: Order of hand-offs between the components

1. **Calibrate (PI).** The source statistics are privatized. Only calibration_stats.json goes to Dataset Generation. The noise log goes to Risk and Compliance and nowhere else.

2. **Generate and encrypt (DG).** CARLA is configured from the privatized values. Each captured file is hashed, access-checked and encrypted.
3. **Decrypt and verify (UE, RC).** Utility Evaluation decrypts the dataset once. Risk and Compliance re-hashes every file against the manifests, and the pipeline stops if any file fails.
4. **Poisoning gate (UE).** Every label file is checked, and only accepted labels move on.
5. **Train and attack (UE, PI).** Utility Evaluation trains YOLOv8 detectors and computes mAP and FID. Privacy Integration trains private and non-private classifiers and attacks them.
6. **Audit and report (RC).** Risk and Compliance runs its checks, builds the compliance report and signs the certificate.
7. **Release decision.** The verified claims, the compliance report and the utility figures are reviewed together.

The path of a single frame illustrates this flow. The first kept frame of Town01 is frame 224, because CARLA drops frames while it warms up. Its four images are hashed and encrypted into files such as rgb_f000224.png.enc. Their digests go into the Town01 manifest under the plaintext names. After decryption, each file is checked against the manifest. The label file bbox_f000224.txt then passes through the poisoning gate. Only then can the frame train a detector or a private classifier. By the time any result based on this frame is reported, three components have checked it.

4 COMPONENT INTEGRATION AND COHESIVE OPERATION

4.1 Integration strategy

We used a file-based integration strategy and agreed on the file contracts before building. Each member built and tested one component alone, but all components share one repository and one set of contracts. Components never call each other directly. They pass files, and every file that crosses a boundary is checked against a JSON Schema contract. Each component could therefore be built on its own machine and joined later without changing its internal code.

We integrated the components in the order that data flows. The calibration hand-off between Privacy Integration and Dataset Generation came first, because no capture could start without it. The encrypted dataset then went to the other components. The clean labels from Utility Evaluation followed, and the signed evidence for Risk and Compliance came last. A pipeline runner, started with the command r26 run, joins the stages in this order. Each stage runs in its component's own Python environment, because their dependencies conflict.

Each interface has one owner, the producer of the file. The producer defines the contract, and the file is validated as soon as it is produced. This approach addressed three integration risks. First, a schema error stops the pipeline at the stage that caused it. It cannot cause a silent error later. Second, each key travels on its own path, so no single component holds every secret. Third, Risk and Compliance checks the others independently, so a fault in one component can be caught by another.

Table 4.1: Integration sequence and the check at each hand-off

Step	Hand-off	Contract owner	Check at the hand-off
1	Privatized calibration statistics, PI to DG	PI	Schema check; the simulator profile is compared with the privatized file
2	Encrypted towns and manifests, DG to UE, PI and RC	DG	Manifest gate: every decrypted file must match its SHA-256 entry
3	Clean labels, UE to PI	UE	Poisoning gate verdict; rejected frames never reach a model
4	Signed evidence and budget logs, PI to RC	PI	HMAC signature and SHA-256 digest must match

Step	Hand-off	Contract owner	Check at the hand-off
5	Privacy summary, PI to UE	PI	Schema check and seeds gate before the joint figure
6	Compliance report and certificate, RC to the release folder	RC	Report contract; certificate fingerprint and signature

Integration happened in three environments. Each member first ran their component on their own machine (Table 5.1). The shared repository then brought the code together. Continuous integration ran all tests and an end-to-end pipeline test on each push. Finally, the full experiment ran on the five-town dataset. Each component worked on files handed over by the one before it. Table 4.1 shows the integration steps and the check at each hand-off.

4.2 Interfaces data contracts and shared models

The shared library holds 16 JSON Schema contracts. Table 4.2 lists the main files passed between components. Appendix A gives the format and validation rule for each one.

Table 4.2: Main files exchanged between components

File	From and to	Purpose
calibration_stats.json	PI to DG and RC	Privatized statistics that configure CARLA, with the total ε spent
calibration_audit_log.json	PI to RC only	Noise per query for the calibration checks; never released
Encrypted towns and manifests	DG to UE, PI and RC	Training frames and integrity digests
clean_labels/	UE to PI	Labels that passed the poisoning gate
audit_report.json and audit_certificate.json	PI to RC	Signed evidence for each training run
summary.json	PI to UE	Attack AUC and test AUC for the joint figure
poisoning_audit.json and utility_results.json	UE to RC and PI	Gate verdict and utility evidence

File	From and to	Purpose
privacy_certificate.json	PI to the release folder	Combined and signed privacy verdict
compliance_report.json	RC to the release folder	Check results, GDPR mapping and overall verdict
pipeline_report.json	Pipeline to the release folder	Integrated verdict of the run

Several models and identifiers are shared between the components.

- **Labels.** All labels use the YOLO format: a class ID and the normalized center, width and height of one object. There are three classes: car, pedestrian and cyclist.
- **Encrypted file layout.** Each file of the delivered dataset holds a 16-byte initialization vector, followed by AES-256-CBC ciphertext with PKCS7 padding.
- **Manifests.** Each town has a SHA-256 manifest of the original plaintext files. A decrypted file can therefore be checked bit for bit.
- **Identifiers.** Frame numbers repeat across towns, so a town prefix is added to every file before the towns are merged. Training runs are named by configuration and seed.
- **Keys.** The AES dataset key and the HMAC signing key are separate. Both are read from local environment files that are excluded from version control.
- **Metadata.** Each town has a CSV file with one row per frame. It names the image and label files and records the object count, weather, time of day, event type and town.

Table 4.3 shows the two calibration releases. The earlier release ran every query at $\varepsilon = 1.8$, which composes to 12.6. The corrected release splits a total of 1.8 over the seven queries. The delivered dataset was captured from the corrected release. The source values and the noise drawn for each query are kept only in the audit log.

Table 4.3: Privatized calibration statistics of the two releases

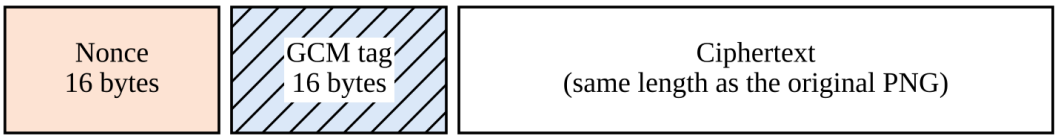
Statistic	Earlier release	Corrected release (used for capture)
Vehicle density per km	2.5139	5.1602

Statistic	Earlier release	Corrected release (used for capture)
Pedestrian density per km	0.9153	0.4806
Average vehicle speed (km/h)	22.0135	30.4335
Average pedestrian speed (km/h)	0.2789	1.9526
Intersections per km	0.2796	−5.2851
Weather: clear, cloudy, rain, fog	0.624, 0.135, 0.241, 0.000	0.650, 0.156, 0.195, 0.000
Time: morning, afternoon, evening, night	0.144, 0.115, 0.575, 0.167	0.000, 0.000, 0.837, 0.163
ε per query and total	1.8 and 12.6	0.2571 and 1.8

The corrected values are much noisier, because each query now has a seventh of the budget. A noised value can fall outside its natural range, such as the negative intersection count. Mapping it into range during simulator setup is post-processing and costs no budget.

The signed evidence of Privacy Integration follows a fixed model. Each audit report records the final ε, the clipping norm and the noise multiplier. It also records δ, the limit and the number of steps. It is signed with HMAC-SHA256, and its SHA-256 digest is stored next to it. For the non-private baseline, a signed declaration states that no privacy mechanism was applied. The budget log has one row per training step.

Dataset Generation can also encrypt with GCM. Figure 4.1 shows that file layout. Each file stores a fresh 16-byte nonce, a 16-byte tag and the ciphertext. It is therefore exactly 32 bytes larger than the original. A consumer of a GCM capture must read the nonce and tag first. If the tag check fails, it writes no output. The delivered dataset uses CBC, so a GCM capture must be agreed with Utility Evaluation before delivery.



Layout of every .png.enc file

Figure 4.1: Layout of an AES-256-GCM encrypted dataset file

Each contract also covers error handling. A failed check stops the stage, and the problem goes back to the producer, who fixes it at the source. When Risk and Compliance found that the calibration total was understated, Privacy Integration changed its own calibration module. Every correction therefore stayed in the component that owns the file.

4.3 Shared nonfunctional requirements

Table 4.4 lists the nonfunctional requirements shared by all components and how we met each one.

Table 4.4: Shared nonfunctional requirements

Requirement	Target	How it was met
Security	No readable data or secrets outside their trusted boundary	Encryption at capture; separate keys outside version control; commit checks that block keys and data
Privacy	Formal guarantee where real data enters	Laplace calibration within a total ε of 1.8 and DP-SGD with a hard limit
Integrity	Any change to data or evidence is detected	SHA-256 manifests, GCM tags and HMAC-signed reports and certificates
Reliability	Errors stop the pipeline at their source	JSON Schema contracts, manifest and seeds gates, budget limit error
Reproducibility	Results can be repeated	Fixed seeds and splits, separate output folders and recorded lineage for every stage
Performance	Protection does not slow the pipeline	About 15 ms to encrypt and 26 ms to decrypt a 4 KB test file
Maintainability	Components can be tested alone	Security and privacy code kept free of simulator code; unit tests on every push
Usability	Results readable by non-technical reviewers	One-page PDF certificates with a verdict, check results and limits

These requirements follow professional standards. SHA-256 and AES follow FIPS 180-4 and FIPS 197 [27], [28]. The evidence is mapped to the GDPR [3].

The security and privacy requirements came from a shared threat model. Dataset Generation wrote it first for the stored files, and the other components adopted it. The simulation host and the pipeline code are trusted. The storage and the people who can reach it are not fully trusted. The adversary can read or change files in storage, or try to use a role that lacks a permission. Table 4.5 lists the threats and the control chosen for each.

Table 4.5: Shared threat model and controls

Threat	Example	Control
Tampering	Images or labels are changed before training	SHA-256 manifest per town; GCM tag; poisoning gate for labels
Disclosure	Dataset files are copied from storage or a stolen disk	AES-256 encryption; plaintext removed after encryption
Privilege misuse	An ML user reads raw simulation output	RBAC gateway with deny-by-default permissions
Duplicate detection	Identical images give identical ciphertext	Fresh random IV or nonce for every encryption
Membership leakage	A model reveals whether a frame was in its training set	DP-SGD with a hard budget limit, tested with attacks
Recovery of source statistics	Published calibration values reveal the real values	Laplace noise on every query; audit log never released
Altered evidence	A signed report is edited after training	HMAC-SHA256 signature checked by Risk and Compliance

Two further requirements are scalability and accessibility. Hashing reads each file in 4,096-byte blocks, so memory use does not grow with file size. Each GCM file grows by only 32 bytes. Because the components exchange files, each can run on a separate machine. For accessibility, the main readers of the evidence are not programmers. Both certificates therefore print the verdict, the result of each check and the known limits on one page.

4.4 Integrated workflow and exception handling

In normal operation, the pipeline runs the seven steps of Section 3.7 in order. Each stage reads only files that earlier stages produced. Every stage also records what it read, what it wrote and a digest of the code that ran. A rerun therefore repeats only the stages whose inputs, settings or code changed. Table 4.6 lists the main failure paths and the response to each.

Table 4.6: Failure paths and responses

Failure	Detected by	Response
A file is missing a field or has the wrong type	JSON Schema contract	The run stops at that stage
A decrypted file does not match its manifest	Manifest gate (RC)	The run stops; no later stage may use the data
An encrypted file is changed (GCM engine)	GCM tag check	Decryption fails and no output file is written
A label line is malformed or a batch's class mix shifts	Poisoning gate	The frame or batch is excluded; verdict FILTERED
Training results hold seeds that were not configured	Seeds gate	The run stops before the summary is used
A user role lacks a permission	RBAC gateway	The request is denied and the decision is printed
The privacy budget reaches its limit	DP training wrapper	Training stops with BudgetExceededError
A signed report is changed	HMAC verification in RC	Verification fails; the compliance verdict is FAIL
A check has not run	Compliance report	Verdict INCOMPLETE, which is not a failure

Recovery is simple, because no stage overwrites its input. The encrypted originals are never deleted after decryption, so decryption can be repeated. Each training run writes to its own folder, so a failed run can be repeated alone. The costly case is a change to the calibration statistics. That needs a new capture, because the privatized values set up the scenes.

Some parts of the pipeline rely on a fallback. The delivered dataset uses CBC, which cannot detect a changed ciphertext by itself. For this dataset, the SHA-256 manifests

are the integrity check, and the manifest gate enforces them. Utility Evaluation had no supported GPU, so its detectors were trained on a CPU. This made training slower but did not affect the comparisons, which all used the same hardware. Opacus secure mode could not be used with current PyTorch versions. Privacy Integration trained without it and declared this in both certificates.

4.5 Team contribution and dependency matrix

Table 4.7 shows what each student did for the integrated solution.

Table 4.7: Team contribution matrix

Student	Research tasks	Development tasks	Integration work	Test evidence
IT22309556 (PI)	Differential privacy, DP-SGD and membership inference	Calibration module, DP training wrapper, attacks, stress test, signed certificate	Calibration hand-off, signed evidence, shared library, contracts and pipeline runner	135 unit tests; attack and stress-test results
IT22541284 (DG)	Secure data generation and authenticated encryption	CARLA capture, SHA-256 manifests, AES engine, RBAC gateway, security audit	Encrypted dataset, manifests and metadata for all components	26 unit tests; six-test security audit
IT22110220 (UE)	Detection utility, domain gap and data poisoning	Decryption, three-layer poisoning gate, YOLOv8 and KITTI pipelines, FID	Clean labels for PI, utility results for PI and RC, joint figure	117 unit tests; cross-domain results; planted-defect tests
IT22066916 (RC)	Privacy auditing, anonymity and GDPR compliance	Seven independent checks, compliance report and signed certificate	Manifest gate; verification of PI, DG and UE evidence	79 unit tests; compliance report and certificate

Table 4.8 shows what each component needs from the others.

Table 4.8: Dependency matrix between components

Component	From PI	From DG	From UE	From RC
PI	none	Encrypted dataset	Clean labels; utility results	Composition finding; manifest gate
DG	Privatized statistics	none	none	none
UE	Privacy summary	Encrypted dataset	none	Manifest gate
RC	Signed evidence and audit log	Manifests and metadata	Poisoning audit and utility results	none

Some outputs belong to the whole group. Table 4.9 lists them and who worked on each. Every member reviewed these outputs before they were used.

Table 4.9: Shared outputs of the group

Shared output	Members	What it is
Shared repository, library and contracts	All four	One code base and one set of file formats for the whole pipeline
Pipeline runner and end-to-end test	All four	Runs the stages in order and checks every hand-off on each push
Joint privacy and utility figure	PI, UE	Combines the privacy summary with the utility results
Compliance report and certificate	RC, with evidence from PI, DG and UE	One verdict built from all checks
Research paper, website and this report	All four	The group outputs that present the integrated solution

We agreed that each member contributed 25% of the project. Appendix B gives the contribution declaration.

5 INTEGRATED IMPLEMENTATION AND PROJECT EXECUTION

5.1 Common technology stack and infrastructure

All components were written in Python and kept in one shared repository. Table 5.1 lists the software and hardware that each component used.

Table 5.1: Development and test environment of each component

Component	Software	Hardware
Privacy Integration	Python 3.11, PyTorch 2.5.1, Opacus 1.4.1, ReportLab	Laptop with NVIDIA RTX 3050 Ti (4 GB), Windows 11
Dataset Generation	CARLA Python API, hashlib, PyCryptodome, NumPy	CARLA capture machine
Utility Evaluation	Python 3.11, Ultralytics YOLOv8, PyTorch	Intel Core i5-8250U CPU
Risk and Compliance	Python 3.11, pandas, PyCryptodome, ReportLab	Laptop CPU; no GPU needed

We made the main shared choices so that the components would work together and their results could be checked. Python was used everywhere, so the shared library could hold the decryption, key loading and label parsing code once. PyCryptodome [46] gave the same AES implementation to the component that encrypts and to those that decrypt. JSON was used for every exchanged file. It can be checked against a schema, and both people and code can read it. GitHub Actions ran the tests on each push. We needed no shared database or server, because the components exchange files.

5.1.1 Software structure of each component

Each component keeps its own folder in the shared repository, but all four follow the same habits. Settings are kept in one place, keys come from the environment and every run writes to its own output folder. Each step has its own command, so it can be run and checked alone. Table 5.2 lists the main modules of each component.

Table 5.2: Main modules and commands of each component

Component	Main modules or commands	What they do
PI	privacy_integration package; pi-calibrate, pi-train, pi-attack, pi-stress, pi-certificate	Privatize statistics, train with DP-SGD and track the budget, run the attacks and the stress test, sign reports and build the certificate
DG	capture_town.py; SecurityEngine and RBACGateway; security_audit.py and notebook	Capture sensor data in each town, hash and encrypt files, check permissions and test the security layer without CARLA
UE	utility_evaluation package; ue-decrypt, ue-detect, ue-train, ue-cross-eval, ue-fid	Decrypt and verify, screen labels, train detectors, run the four experiments and compute FID
RC	risk_compliance package; rc-verify-certs, rc-audit-budgets, rc-reconstruct, rc-report, rc-certificate	Run the independent checks, build the compliance report and sign the certificate

Privacy Integration is an installable package. All of its settings live in config.yaml, and conflicting values are rejected at start-up. The DPTrainingWrapper class attaches an Opacus PrivacyEngine to the model, optimizer and data loader. It finds the noise multiplier for the target budget and reads the accountant after every step. If the cumulative ε reaches the limit, it raises BudgetExceededError and stops the loop.

Dataset Generation keeps its two security classes free of simulator code. They therefore run in both the capture script and the security audit. One capture script serves all five towns, and the differences between towns are kept in a table of profiles. Hashing reads each file in 4,096-byte blocks. The encryption method writes the encrypted copy first and removes the plaintext only afterwards. Its core is short:

```
def encrypt_bytes(self, data):
    if self.mode == "gcm":
        cipher = AES.new(self.key, AES.MODE_GCM,
nonce=os.urandom(NONCE_BYTES))
        ciphertext, tag =
cipher.encrypt_and_digest(data)
        return cipher.nonce + tag + ciphertext
    cipher = AES.new(self.key, AES.MODE_CBC)
    return cipher.iv + cipher.encrypt(pad(data,
AES.block_size))
```

A missing or wrong-length key raises an error. The code does not silently create a random key, because that would make the dataset unrecoverable.

Utility Evaluation used YOLOv8n, the smallest member of the YOLOv8 family. Both detectors were trained for up to 50 epochs at 640 × 640 pixels. The batch size was 8 and the seed was 42. Early stopping used a patience of 15 epochs, and the best checkpoint was kept by validation mAP. The KITTI labels were converted to the same three classes. Car, Van and Truck became car, Pedestrian and Person_sitting became pedestrian, and Cyclist stayed cyclist. Tram, Misc and DontCare were left out.

Risk and Compliance wrote each check from the published file formats alone. It reuses no signer, hash routine or noise function from the component it checks. Each check writes one JSON result with its own verdict. The report builder combines them. The certificate module then prints a one-page PDF with a fingerprint, a signature and a QR code [47].

5.2 Development and integration process

As a group, we followed an Agile approach based on SCRUM. Each member built a separate component, but all components had to connect into one pipeline. We planned the work in two-week sprints with a shared backlog. Each sprint covered planning, design, implementation and testing, and ended with a review with the supervisors. Figure 5.1 shows this cycle.

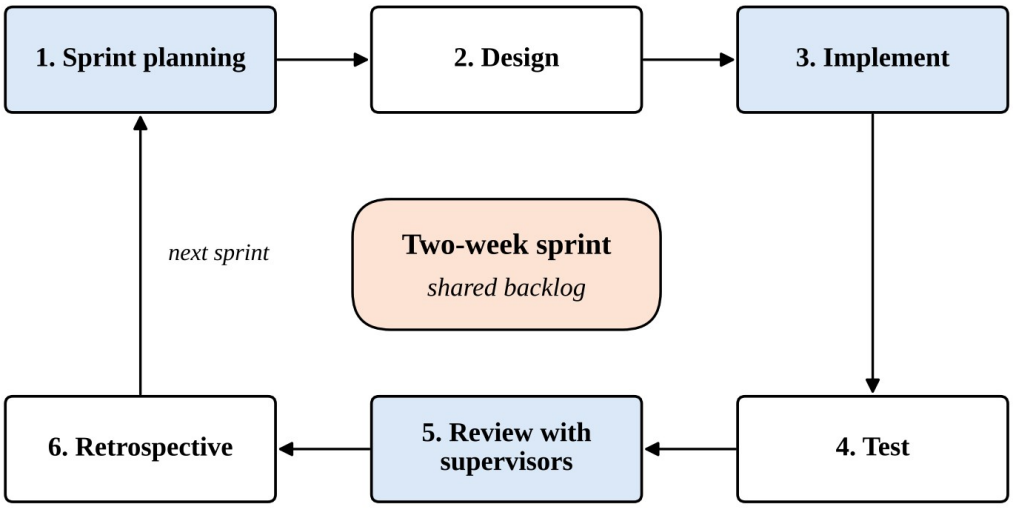


Figure 5.1: Agile SCRUM cycle followed by the group

Each member first developed in their own working folder. The working code was then brought into one shared repository, with a shared library, the contracts and a pipeline runner. Scripts that depended on one machine's folder layout were reorganized so that their logic could be tested. For example, five per-town capture scripts became one script with a table of town profiles.

Interface contracts were central to the process. Continuous integration runs three jobs on each push. The first runs commit checks, which block keys, environment files, datasets, model weights and unprivatized statistics. The second runs the tests of every component in its own environment. The third runs the whole pipeline on an encrypted fixture and expects an overall PASS. Before the full experiment, a code review found the AES key and the HMAC key written into source files. We replaced both keys and moved them to environment files. A regression test now fails if an old key appears in the source again.

The Agile approach let the requirements change as results came in. Three changes were the most important. The privacy classifier changed after a label problem appeared in the first training runs. The calibration module changed after the composition check found an understated budget. After the second progress review, each component added tests that show its checks can detect a real problem.

5.2.1 Problems found during integration

Joining four components exposed problems that no single member would have encountered alone. Table 5.3 lists the main ones, who found them and how they were fixed.

Table 5.3: Problems found during development and integration

Problem	Found by	Fix
Encryption and signing keys written in source files	Code review	Both keys replaced and moved to environment files; regression test against old keys
Most positive labels were tiny pedestrians, and frames were stretched to a square	PI	Aspect ratio kept; frames with only small targets excluded
Same frame numbers in different towns overwrote each other on merge	UE	Town prefix added to every file name
Many frames have empty label files	UE	All scripts treat empty files as valid frames
Calibration budget reported as 1.8 while the queries spent 12.6	RC composition check	Module now splits a total of 1.8 over the queries; check passes on the new release
Noise log field named epsilon_used held the noise scale	RC	Check infers what the field holds; logs now record ε and scale explicitly
k-anonymity check could not find the metadata columns	RC	Quasi-identifiers set to the column names used by DG; a missing column raises an error
CBC cannot detect a changed ciphertext	DG audit	GCM option added; manifest gate enforced for the CBC dataset

The label problem took the most time. In the first runs, both private and non-private models predicted the majority class for every input. A search over learning rates,

optimizers and batch sizes gave three findings. Adam was unstable under DP-SGD, so private training moved to SGD with momentum. Loss-based class weights did not work well with per-sample clipping. Opacus replaces a weighted sampler, so balancing had to be done by duplicating frames. None of these changes fixed the main problem. A label check then looked at the positive frames. In 62.3% of them, the largest pedestrian was smaller than about 8 × 8 pixels after resizing. The frames were also stretched into a square. The aspect ratio was then kept, and frames with only small targets were excluded. The baseline test AUC rose from about 0.50 to 0.84. The early failure came from data preparation, not from differential privacy.

The budget problem shows the value of separate checks. Each of the seven calibration queries had used $\varepsilon = 1.8$, so the queries composed to 12.6. The calibration file, however, reported a total of 1.8. The noise of each query had the correct size, so a per-query check could not see the fault. Only the composition check, written by another member, found it. Section 6.3.5 gives the details.

Other problems were practical. The default output folders of Ultralytics are nested, so checkpoint paths had to be checked before cross-evaluation. About 12 GB of KITTI images had to be downloaded on limited disk space. CPU-only training meant overnight runs, so the computer's sleep settings had to be changed.

5.3 Deployment and operational configuration

The integrated solution runs as a series of command-line stages joined by the pipeline runner. The command r26 check confirms the environments, inputs and keys. The command r26 plan lists every stage and whether it will run. The command r26 run runs whatever is not up to date. The command r26 validate rechecks every hand-off file against its contract. Keys come from environment variables or a local .env file, so the same code runs on any machine.

Monitoring relies on the evidence files. Every training run writes a budget log and a signed report. The poisoning gate writes a verdict, and every compliance check writes a JSON result. The release stage collects the privacy certificate, the utility results and the compliance report. The integrated verdict is written to pipeline_report.json. It is PASS when both certificates pass and all four utility experiments ran. It is FAIL when

a privacy or compliance verdict failed, and INCOMPLETE otherwise. For recovery, the encrypted originals stay unchanged and every run writes to its own folder. Any stage can therefore be repeated from its inputs.

Table 5.4 shows where each stage runs in the intended deployment, what it reads and what it writes. Only privatized, encrypted or signed files cross from one machine to another.

Table 5.4: Stages of the deployed pipeline

Stage	Runs on	Reads	Writes
Calibration	Data owner's machine	Real aggregate statistics	calibration_stats.json; audit log for RC
Capture and protection	CARLA machine	Privatized statistics	Encrypted town folders, manifests and metadata
Decryption and gates	Analysis machine	Encrypted towns, manifests, labels	Decrypted images, manifest result, clean labels
Detector training	Analysis machine	Decrypted images and labels; KITTI	Checkpoints, mAP results, FID
Private training and attacks	Analysis machine	Decrypted images and clean labels	Models, budget logs, signed reports, certificate
Independent audit	Auditor's machine	Signed evidence, audit log, manifests, metadata	Check results, compliance report, certificate

To set up a stage, the operator installs the component's requirements in its own environment. The needed keys go in a local .env file. The Dataset Generation audit runs on any machine without CARLA. During the project, we backed up the encrypted dataset and the keys to secure cloud storage.

5.4 Evidence of team execution

Our timeline started with topic selection and feasibility, then the literature review, the proposal, and requirement analysis and design. Component implementation, integration, testing and experiments followed. The last stage covered the research paper, the website, the final reports and the final presentation. Figure 5.2 shows the timeline, and Figure 5.3 shows the work breakdown into four component streams.

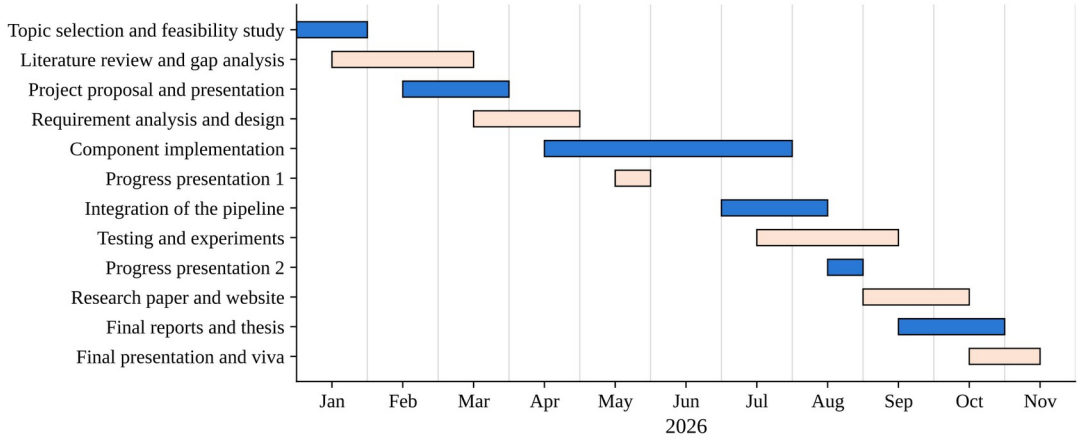


Figure 5.2: Gantt chart of the project

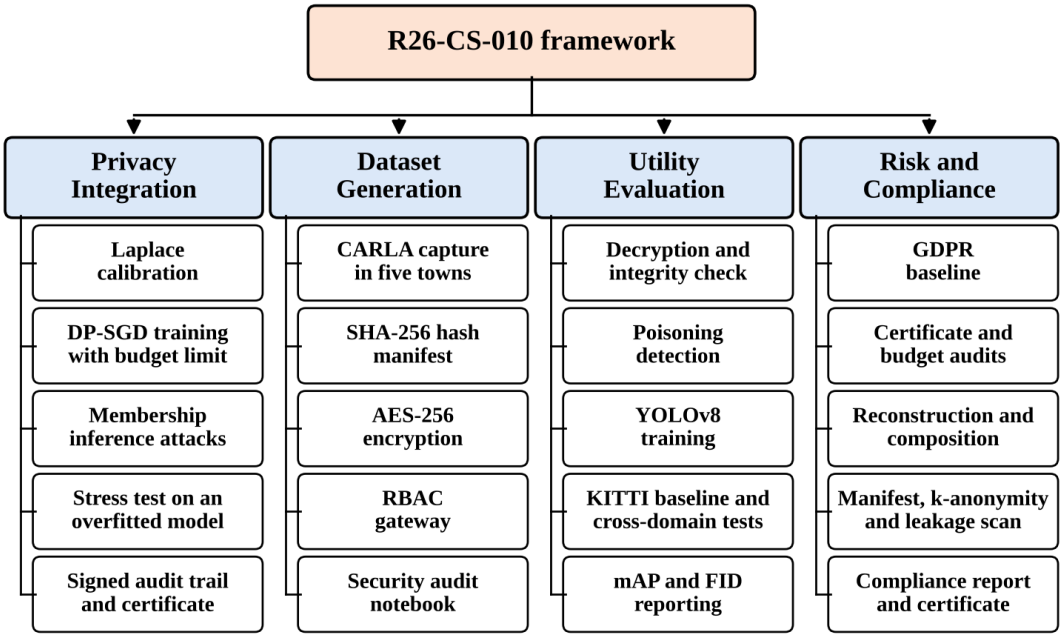


Figure 5.3: Work breakdown structure of the project

The main evidence of how the team carried out the work is listed below.

- **Version control and continuous integration.** All components live in one repository, and every push runs the commit checks, the component tests and the end-to-end test.
- **Supervisor reviews and progress reports.** Each sprint ended with a review with the supervisors. Members also kept research logbooks and wrote monthly progress reports.
- **Progress reviews.** The proposal and progress review panels reviewed the project. We recorded their comments and responded to them (Section 6.4).
- **Cross-component checks.** Risk and Compliance verified the outputs of the other three components with its own code. Its compliance report records the result of each check.

Each of the four streams covered the same five kinds of work: research, implementation, testing, result collection and documentation. This made progress simple to compare at each review. The integration outputs are also evidence. The signed reports, budget logs, manifests, gate verdicts and compliance results were each checked by a second member. The problems in Table 5.3 were found and fixed through this shared work.

6 INTEGRATED TESTING EVALUATION AND VALIDATION

6.1 Evaluation framework and success criteria

The group evaluation asked four questions, one for each subproblem:

- **EQ1 (SP1):** Does differential privacy at calibration and training stop membership inference, and at what cost?
- **EQ2 (SP2):** Are the generated files confidential and tamper-evident from the moment of capture?
- **EQ3 (SP3):** Is the protected synthetic data still useful for training a detector, and are its labels untouched?
- **EQ4 (SP4):** Do the privacy and security claims hold when an independent party checks them?

The group objectives in Table 2.3 served as the acceptance criteria. We also used four baselines or controls. A non-private model was the positive control for every membership inference attack. If a private model then scored near chance, a broken attack could not be the reason. A detector trained on real KITTI images [1] was the utility baseline. Labels with planted defects were the control for the poisoning gate. A deliberately changed report was the control for the signature check.

The main dataset was the five-town CARLA dataset of 10,000 frames. Each frame has an RGB image, a depth map, a semantic segmentation image and a class mask. The real baseline was the KITTI training split of 7,481 images. A separate Town01 capture of 6,093 images tested the GCM engine on real output. No human participants took part. Privacy results are means and standard deviations over three seeds, with paired t-tests against same-seed baselines. Utility is reported as mAP50, mAP50-95 and utility retention. The individual theses describe each method in full.

Table 6.1 shows how each part of the evaluation used the data.

Table 6.1: Datasets and splits used in the evaluation

Dataset	Use	Size or split
Five-town CARLA dataset	Main synthetic dataset for all components	10,000 frames, 800 × 600 pixels, four images per frame

Dataset	Use	Size or split
Utility Evaluation split	Training and testing the synthetic detector	7,000 / 1,500 / 1,500 (seed 42)
KITTI training split	Real-data baseline for utility	7,481 images; 5,236 / 1,122 / 1,123
Privacy Integration split	Private and non-private classifiers	5,493 / 1,176 / 1,179 usable frames
Stress-test sets	Overfitted targets for the stress test	500 members and 500 non-members, five seeds
Town01 GCM capture	Real-output test of the GCM engine	6,093 encrypted images

Table 6.2 lists the measures. Most come from the cited literature. We chose them so that each question has at least one measure that a reader outside the team can repeat.

Table 6.2: Measures used in the evaluation

Measure	Question	Meaning
Test AUC	EQ1	How well the privacy classifier separates frames with and without pedestrians or cyclists
Attack AUC and success	EQ1	How well an attacker separates members from non-members; 0.5 and 50% mean random guessing
TPR at 1% and 5% FPR	EQ1	Whether the attacker can identify a few members with high confidence
Advantage removed	EQ1	Drop in attack AUC as a share of the baseline advantage over 0.5
Chi-square and bit change	EQ2	Whether the ciphertext looks like random data
Tamper and folder checks	EQ2	Whether changed files are rejected and stored files are unreadable
mAP50 and mAP50-95	EQ3	Detection quality at 50% IoU and averaged from 50% to 95% IoU
Utility retention	EQ3	Synthetic-to-real mAP as a share of real-to-real mAP
FID	EQ3	Distance between the synthetic and real image sets
Check verdicts	EQ4	PASS, FAIL or INCOMPLETE for each independent check and overall
Calibration ratio	EQ4	Observed reconstruction error divided by the error a correct Laplace mechanism gives

6.2 Integration and end to end testing

We tested at four levels.

- **Unit tests.** Each component has its own suite. Table 6.3 lists them, and all passed.
- **Interface and system tests.** Every exchanged file is checked against its contract when it is produced. An end-to-end test runs every real stage on an encrypted fixture and expects an overall PASS. It runs on every push.
- **Security tests.** The Dataset Generation audit tested round-trip correctness, nonce uniqueness, ciphertext randomness, bit change, tamper detection and a real captured folder. The Risk and Compliance checks tested the evidence of the whole pipeline.
- **Performance and usability.** Encrypting a 4 KB file took about 15 ms, and decrypting it took about 26 ms. Both certificates print their verdict and limits on one page for non-technical readers.

Table 6.4 lists the main integration and system tests. Most check a hand-off between two components. Some check a property that a later component depends on.

Table 6.3: Automated test suites of the framework

Suite	Tests	Main areas covered
Shared library	50	Decryption and manifest checks, key loading, every hand-off contract
Pipeline	37	Stage graph, gates, run state and the integrated report; plus the end-to-end run
Privacy Integration	135	Laplace noise and composition, budget halt, attacks, stress test, signing and certificate
Dataset Generation	26	Security engine, security audit and capture logic with CARLA stubbed
Utility Evaluation	117	Poisoning gate against injected defects, size-stratified AP, FID sampling
Risk and Compliance	79	Each check with a planted fault, report verdicts and certificate verification

Table 6.4: Integration and system test results

Test	Expected result	Actual result
Manifest gate	Every decrypted file matches its digest	40,000 of 40,000 files matched
Poisoning gate on the full dataset	Structural faults rejected; outlier batches excluded	0 structural faults; 24 batches excluded; FILTERED
Poisoning gate on planted defects	Out-of-range box, unknown class and relabeled batch caught	All caught in tests
Calibration reaches the simulator	Town01 profile equals the privatized file	Matched field by field
Budget limit	Training halts at the limit	Halted in a low-limit test
Per-step budget logging	One row per step	3,090 rows per private run
Repeatability	Same seed gives the same result	Same loss and test AUC
Signed report tampering	Changed report is rejected	Rejected by PI and by RC
End-to-end pipeline	Every stage runs on the fixture	Overall PASS

We did not run a formal user acceptance test with external users. Section 6.4 describes how the solution was reviewed instead.

6.3 Overall results

Table 6.5 summarizes the main result of each component. The sections after it give the group-level evidence. The detailed experiments are in the individual theses.

Table 6.5: Summary of the main results of each component

Component	Main result	Answer to its question
Privacy Integration	Attack AUC fell from 0.639 to 0.504 to 0.515 (ε = 1, 3, 8), at 0.16 to 0.21 test AUC; ε = 1 removed 99.5% of the advantage in the stress test	EQ1: yes, at a measured cost
Dataset Generation	All audit tests passed; a changed byte was rejected; 0 of 6,093 captured files readable	EQ2: yes
Utility Evaluation	mAP50 of 0.3655 on synthetic images; 10.46% retention on KITTI; FID 115.99; gate FILTERED	EQ3: useful within the synthetic domain, weak on real images

Component	Main result	Answer to its question
Risk and Compliance	All checks passed on the final outputs; 8 of 8 GDPR provisions verified; one budget fault found and fixed	EQ4: yes, within the scope of the checks

6.3.1 Privacy at calibration and training

The calibration profile in the Town01 configuration matched the privatized file field by field. The simulator therefore used the privatized values. The Town01 metadata confirms this at the frame level. Of its 2,000 frames, 1,248 are clear, 270 are cloudy and 482 are rainy. These shares equal the privatized weather values of the release used for capture: 0.624, 0.135 and 0.241.

For training, Table 6.6 gives the mean over three seeds for each configuration. Figure 6.1 plots the two main measures.

Table 6.6: Privacy and utility by configuration (mean ± standard deviation over three seeds)

Configuration	Target ε	Test AUC	Attack AUC	Attack success	Loss gap
No DP	none	0.829 ± 0.009	0.639 ± 0.013	56.7%	0.360
Config A	1.0	0.621 ± 0.045	0.504 ± 0.002	50.3%	0.001
Config B	3.0	0.655 ± 0.035	0.515 ± 0.002	51.6%	0.013
Config C	8.0	0.668 ± 0.020	0.506 ± 0.007	50.8%	0.026

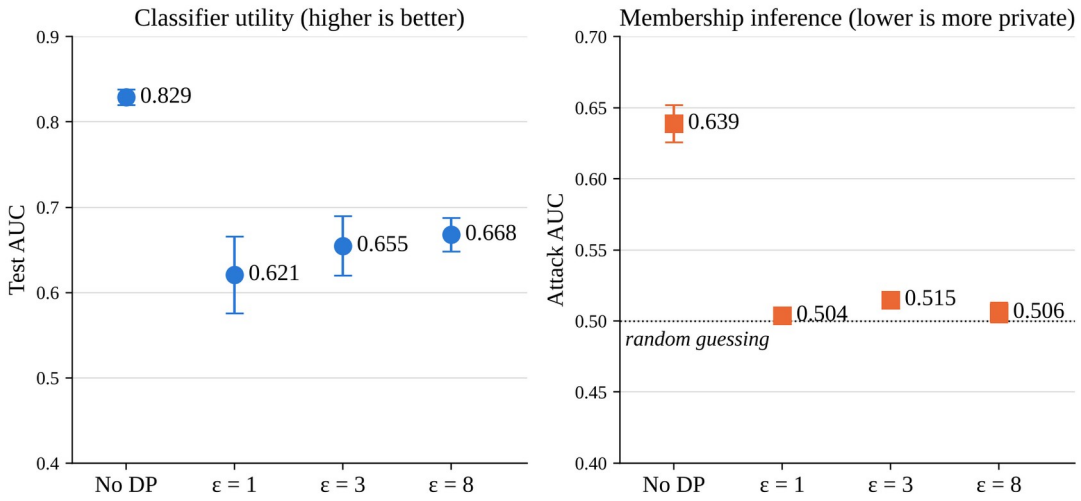


Figure 6.1: Test AUC and attack AUC by privacy configuration (mean and one standard deviation over three seeds)

The attack worked against the non-private baseline, with an AUC of 0.639 and a success rate of 56.7%. The baseline loss on training images was 0.36 lower than on unseen images, and the attack uses this gap. Against every private configuration, the attack AUC fell to between 0.504 and 0.515. The success rate fell to between 50.3% and 51.6%, close to random guessing. Single runs did not overlap either. The nine private runs had an attack AUC of 0.499 to 0.516. The three baseline runs had 0.625 to 0.650. The cost was a fall in test AUC from 0.829 to between 0.621 and 0.668.

Each private configuration was compared with the baseline trained with the same seed. Paired t-tests showed that both drops were significant at the 0.05 level (Table 6.7). With three seeds, the differences between the three budgets were not significant.

Table 6.7: Two-sided paired t-tests of each private configuration against the baseline (three seeds)

Configuration	Drop in test AUC	p-value	Drop in attack AUC	p-value
Config A	0.208	0.0099	0.134	0.0025
Config B	0.174	0.0084	0.124	0.0032
Config C	0.160	0.0031	0.132	0.0031

A second attack checked this result. The loss-threshold attack [21] needs no shadow model. It treats a lower loss on the target as a sign of membership. Table 6.8 shows the results, and Figure 6.2 compares the two attacks.

Table 6.8: Loss-threshold attack (mean ± standard deviation over three seeds; random guessing gives an AUC of 0.5 and a TPR equal to the FPR)

Configuration	Attack AUC	TPR at 1% FPR	TPR at 5% FPR
No DP	0.610 ± 0.026	0.007 ± 0.007	0.057 ± 0.012
Config A	0.501 ± 0.007	0.015 ± 0.002	0.053 ± 0.011
Config B	0.503 ± 0.012	0.016 ± 0.003	0.055 ± 0.013
Config C	0.506 ± 0.012	0.014 ± 0.006	0.053 ± 0.009

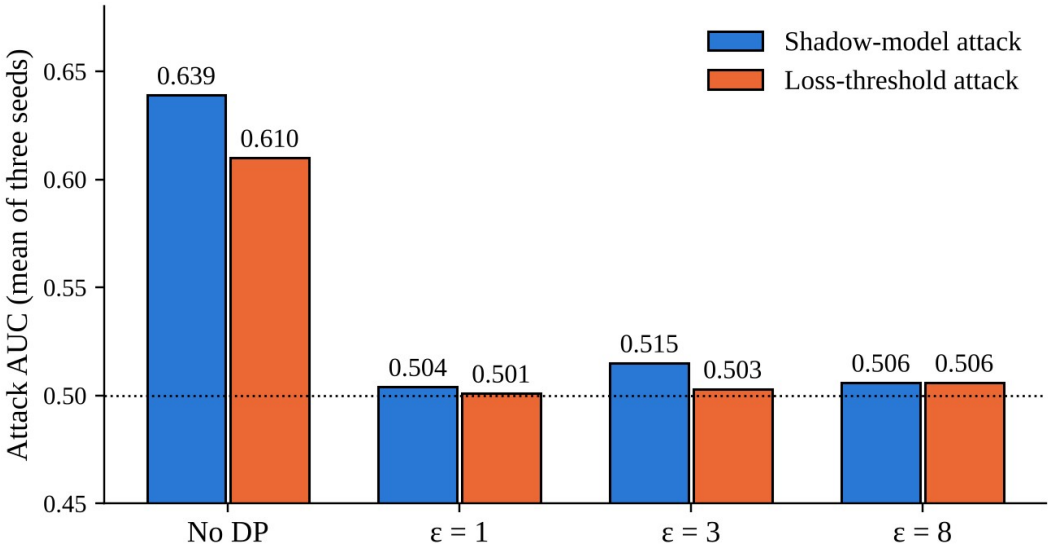


Figure 6.2: Attack AUC of the shadow-model and loss-threshold attacks

The loss-threshold attack gave an AUC of 0.610 against the baseline and 0.491 to 0.518 against the private runs. The paired drop of 0.10 to 0.11 was significant for each private configuration. At low false-positive rates, however, the attack was at chance for every configuration, including the baseline. The baseline therefore leaked only a small average shift in loss. No single frame could be picked out with high confidence, even without DP.

The second progress review panel noted that the drop looked small. The main study could not answer this, because its baseline leaked only moderately. Privacy Integration therefore ran a stress test. Each target was trained on 500 member frames for 60 epochs, so the non-private model memorized its members. Five seeds were used, and the loss-threshold attack was applied. Table 6.9 and Figure 6.3 give the results.

Table 6.9: Stress test on an overfitted model (mean over five seeds; one-sided paired t-tests against the non-private model)

Configuration	Attack AUC	Drop in attack AUC	p-value	Advantage removed
No DP	0.628			
ε = 1	0.501	0.128	0.00004	99.5%
ε = 3	0.526	0.102	0.00006	79.4%
ε = 8	0.547	0.081	0.001	63.0%

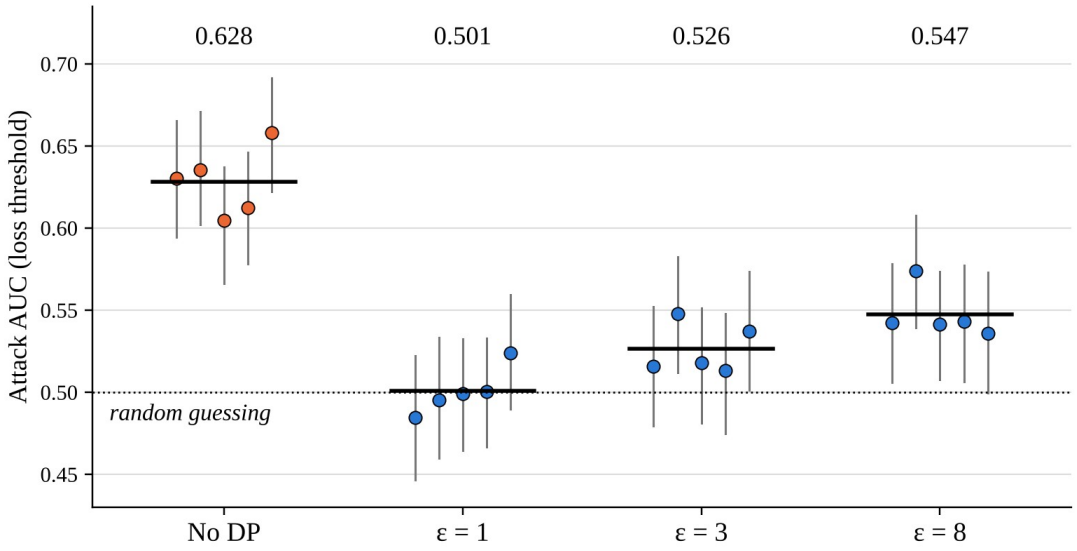


Figure 6.3: Stress test: attack AUC of every seed with its 95% bootstrap interval; the black line marks the mean of each configuration

In this setting, the non-private model showed clear leakage, and the 95% bootstrap interval of every seed lay above 0.5. DP-SGD reduced the attack at every budget, and every drop was significant. At ε = 1, the attack AUC fell to 0.501, which removed 99.5% of the attacker's advantage. The budgets also separated in the order that theory predicts. The step from ε = 1 to ε = 3 was significant (p = 0.013). So was the step from ε = 3 to ε = 8 (p = 0.010). The first version of the stress test used stricter verdict rules. They were replaced by standard tests after the first run. The privacy certificate reports both verdicts.

6.3.2 Protection of generated files

The Dataset Generation audit runs on the real SecurityEngine. Its first five tests use synthetic bytes and a random key that exists only in memory. The sixth test checks a real captured folder with the project key and writes no plaintext. Table 6.10 summarizes the results for the GCM engine, and Figure 6.4 shows four of them.

Table 6.10: Summary of the Dataset Generation security audit (AES-256-GCM)

Property	Evidence	Outcome
Round-trip correctness	Bytes identical after decryption	Pass
Nonce uniqueness	30 of 30 unique nonces and ciphertexts	Pass
Byte distribution	Chi-square 272.59 (critical value 293.25)	Pass
Ciphertext change	50.0% of bits differ	Pass

Property	Evidence	Outcome
Tamper detection	Changed file rejected; no output written	Pass
Real folder	0 of 6,093 encrypted files readable	Pass

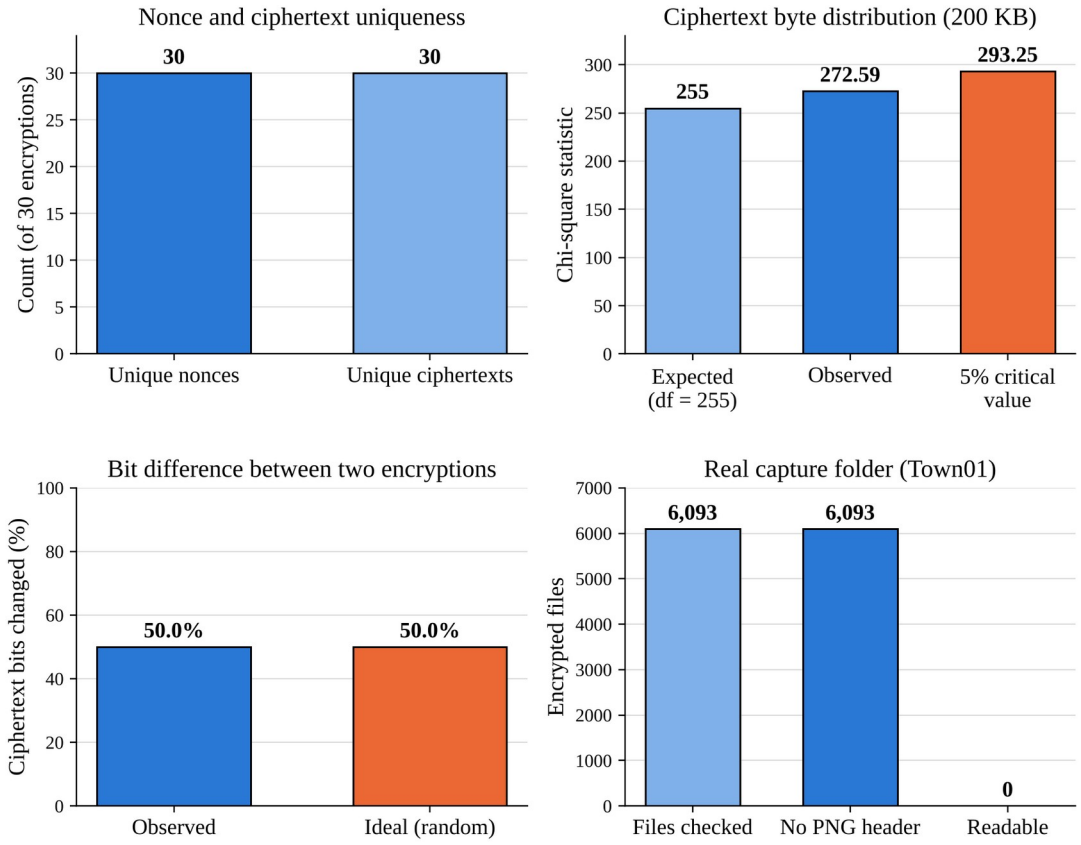


Figure 6.4: Results of the Dataset Generation security audit

Encrypting a 4 KB file and decrypting it gave back the exact original bytes. Encrypting the same file 30 times gave 30 different nonces and 30 different ciphertexts. For the randomness test, the counts of the 256 byte values were compared with a uniform spread. The chi-square value of 272.59 is below the 5% critical value of 293.25. The test therefore cannot tell the ciphertext from random bytes. Two inputs that differ in one bit gave ciphertexts in which 50.0% of the bits differed. This comes from the fresh nonce used for each encryption, which is the property the pipeline needs. In the tamper test, one flipped byte made decryption fail with "MAC check failed", and no output was written. Finally, none of the 6,093 files in the real Town01 folder started with a PNG signature. Each GCM file costs only 32 extra bytes, or about 190 KB for the whole capture.

The delivered five-town dataset uses CBC, which has no tag. For CBC, the audit reports the tamper test as not applicable. Integrity of the delivered dataset therefore rests on the SHA-256 manifests. The manifest gate checked them for every file (Section 6.3.4).

6.3.3 Utility of the synthetic data

Utility Evaluation decrypted all 40,000 image files of the five towns, and every digest matched its manifest. The poisoning gate then checked 10,000 frames and 15,084 boxes. It found no structural faults. The batch layer excluded 24 batches of 100 frames, or 2,400 frames, and the verdict was FILTERED. The cross-town layer warned about Town01, whose cyclist share was 0.030 against 0.116 elsewhere. Figure 6.5 shows the frames kept and excluded in each town.

The gate cannot tell whether an excluded batch was tampered with or simply came from an unusual stretch of driving. It excludes both. Its tests show that it catches real tampering. In planted-defect tests, it caught an out-of-range box, an unknown class and a fully relabeled batch. The detector results below come from training runs on all 10,000 frames, made before the batch layer was added. Retraining on the filtered labels is future work.

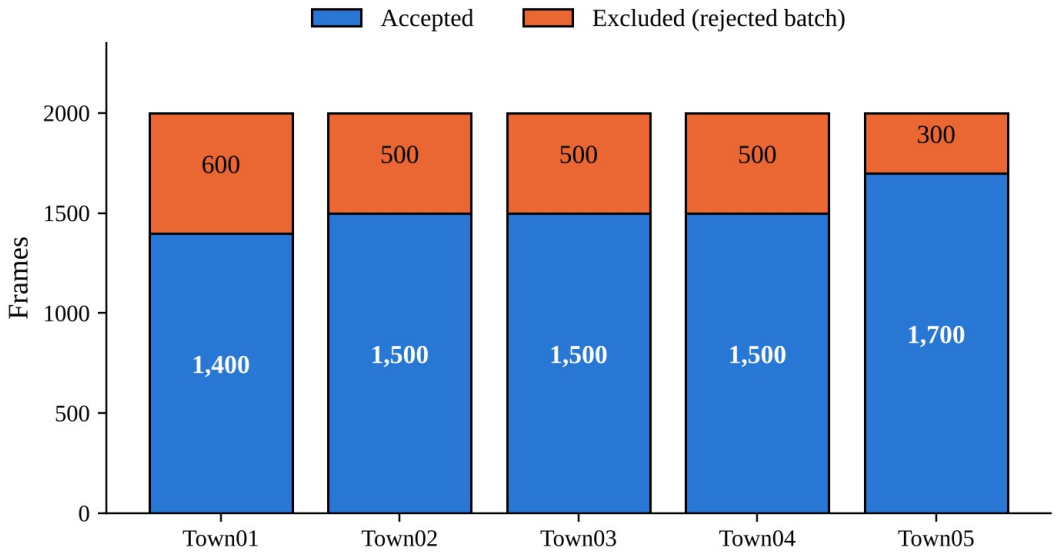


Figure 6.5: Frames accepted and excluded by the poisoning gate in each town

The label files are highly imbalanced. There are 11,008 cars, 2,424 pedestrians and 1,652 cyclists, and 32.6% of frames contain no objects. Pedestrians and cyclists are

also small. Of their boxes, 86.4% and 67.3% are under 32 × 32 pixels. Figure 6.6 shows these findings. They explain why we report per-class results.

Table 6.11 gives the results of the four cross-domain experiments, and Figure 6.7 compares them.

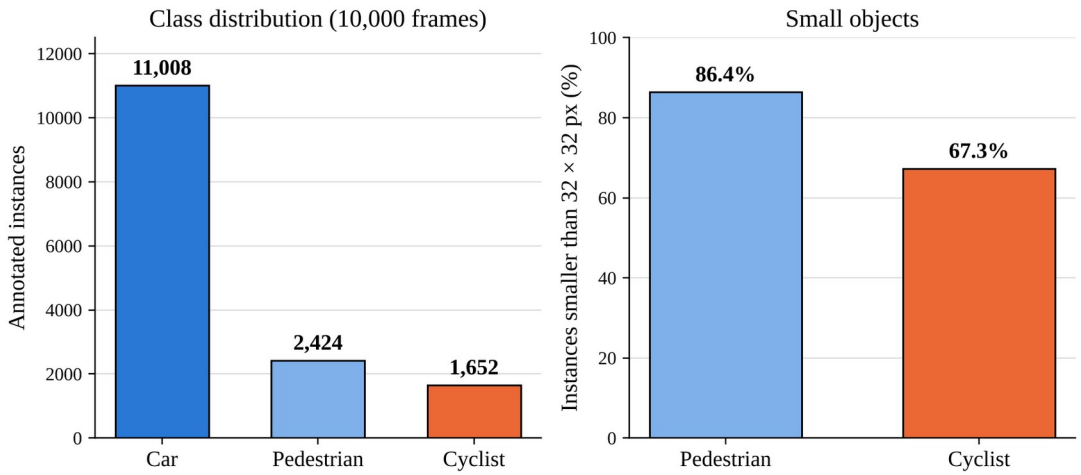


Figure 6.6: Class distribution and share of small objects in the synthetic dataset

Table 6.11: Cross-domain evaluation results

Experiment	mAP50	mAP50-95	Car AP50	Pedestrian AP50	Cyclist AP50
Synthetic to Synthetic	0.3655	0.2273	0.6076	0.1793	0.3096
Synthetic to Real (KITTI)	0.0889	0.0343	0.2614	0.0051	0.0001
Real (KITTI) to Real (KITTI)	0.8500	0.5531	0.9519	0.7769	0.8213
Real (KITTI) to Synthetic	0.0507	0.0199	0.1273	0.0079	0.0171

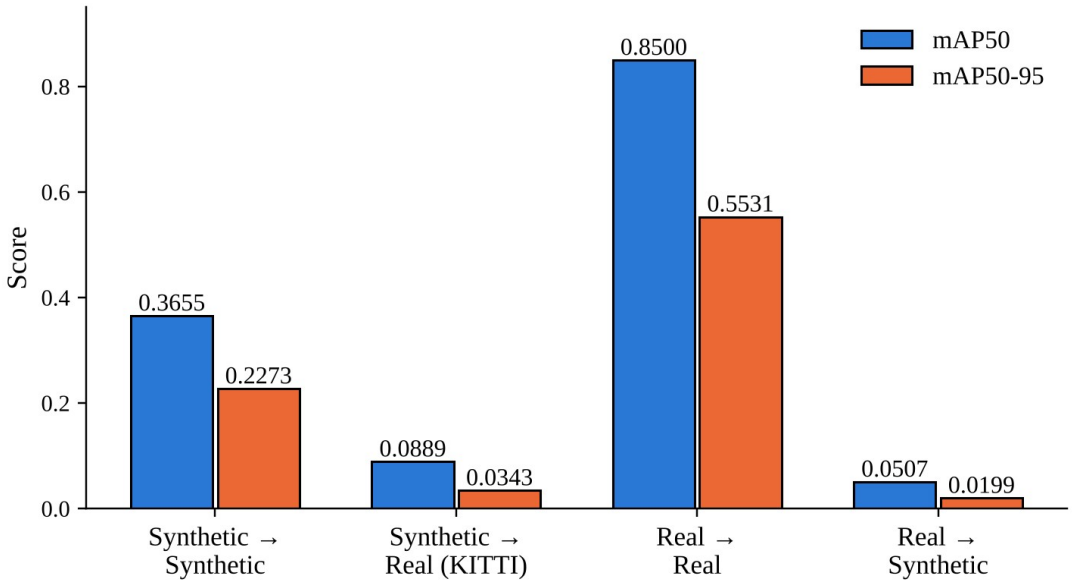


Figure 6.7: mAP50 and mAP50-95 of the four cross-domain experiments

On its own test split, the synthetic model reached an mAP50 of 0.3655. Its synthetic-to-real mAP50 of 0.0889 is 10.46% of the real-to-real mAP50 of 0.8500. On the stricter mAP50-95, the retention is 6.2%. This is the measured cost at the data level. The loss is uneven across classes, as Figure 6.8 shows. Cars kept about 27.5% of their real-data AP50, while pedestrians and cyclists almost disappeared. The reverse transfer gave an mAP50 of only 0.0507, so the domain gap works in both directions. The four results fall in the expected order, which supports the protocol.

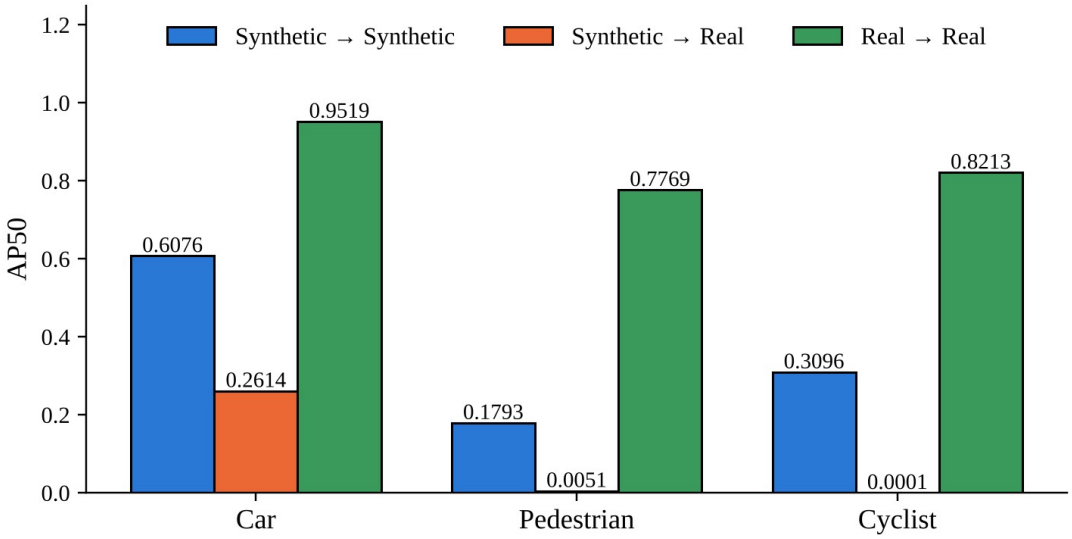


Figure 6.8: Per-class AP50 for training and testing within and across domains

The FID between 1,000 synthetic and 1,000 KITTI test images was 115.99. This falls in the moderate range of Table 6.12. The utility result was much worse than moderate. FID alone does not predict detection quality, so the two measures should be read together.

Table 6.12: FID interpretation scale

FID range	Interpretation
Below 50	Reasonable similarity
50 to 200	Moderate difference
Above 200	Very different

6.3.4 Independent verification

Risk and Compliance ran its checks on the final outputs. Table 6.13 lists the results.

Table 6.13: Results of the independent checks on the final outputs

Check	What was tested	Result	Verdict
Certificate verification	Every signed audit report verifies	12 of 12 reports (4 configurations × 3 seeds)	PASS
Tamper test	A changed report is rejected	ε changed from 0.990572 to 0.01 was caught	PASS
Budget audit	Reported ε equals the last logged ε; never falls; at most 10	9 private runs agree; 3 baselines declared	PASS
Calibration log	Source value plus logged noise equals the published value	5 of 5 scalar fields	PASS
Composition	Queries compose to the claimed total	7 × 0.2571 = 1.8	PASS
Reconstruction	Noise has the size the mechanism requires	Ratio 0.94	Consistent
Manifest verification	Every decrypted file matches the manifest	40,000 of 40,000	PASS
k-anonymity	Smallest group of frames sharing all quasi-identifiers	k = 5 over 192 groups; no unique rows	PASS
Leakage scan	No paths, addresses or credentials in released files	9 files, 0 findings	PASS

The final logged ε values were 0.990572, 2.99984 and 7.995129, against targets of 1, 3 and 8. Each is slightly below its target. Opacus picks the noise so that the last step does not pass the target. Figure 6.9 compares them with the limit.

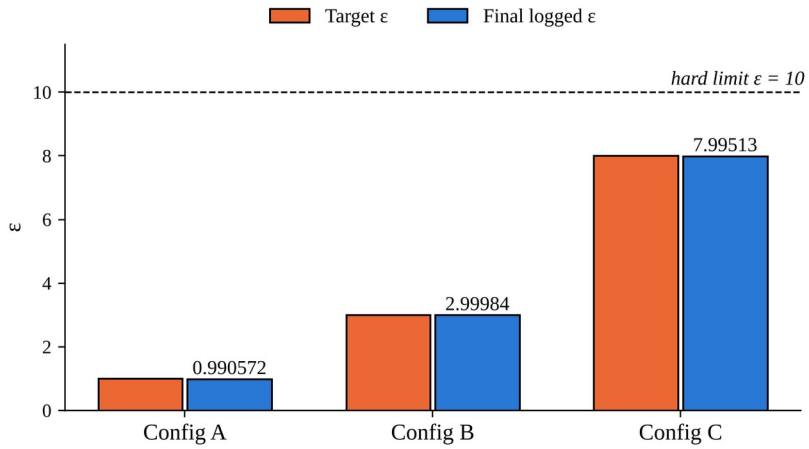


Figure 6.9: Target and final logged ε of the three private configurations

On the corrected calibration release, the mean absolute difference between published and source values was 4.37. A correct Laplace mechanism at ε ≈ 0.2571 gives 4.67. The ratio of 0.94 means the noise had the size that the budget requires. The k-anonymity result holds exactly at the threshold of 5, so the margin is small.

The compliance report combined six of these checks with the poisoning verdict. All seven passed, and all eight GDPR mappings were verified. Table 6.14 shows the mapping.

Table 6.14: GDPR mapping in the final compliance report

GDPR provision	Requirement	Evidence check	State
Art. 5(1)(b)	Purpose limitation	Calibration audit log	Verified
Art. 5(1)(c)	Data minimization	Calibration audit log	Verified
Art. 5(2)	Accountability	Certificate verification	Verified
Art. 17	Right to erasure	k-anonymity	Verified
Art. 25	Data protection by design	Budget audit	Verified
Art. 25	Calibration budget accounting	Calibration composition	Verified
Art. 32	Security of processing	Manifest verification	Verified

GDPR provision	Requirement	Evidence check	State
Art. 32	Integrity of processed data	Poisoning detection	Verified



Figure 6.10: Compliance certificate issued for the final outputs

The overall verdict was PASS, with two declared weaknesses. Opacus secure mode was off, and the earlier calibration releases had understated their total ε. The report states that it covers only the listed checks and is not a full GDPR assessment. It was fingerprinted with SHA-256, signed with HMAC-SHA256 and printed as certificate R26-RC-F3335EAE8286 (Figure 6.10). The verify command confirmed that the report

and the PDF match the certificate record. The signature line is left blank for a handwritten signature.

6.3.5 Combined view

Table 6.15 puts the two cost measurements side by side.

Table 6.15: Cost of privacy at the data level and the model level

Level	Component	Finding
Data	Utility Evaluation	A detector trained on the synthetic data keeps 10.46% of the real-data mAP50 on real KITTI images
Model	Privacy Integration	DP-SGD lowers test AUC by 0.16 to 0.21 and brings membership inference to chance

At the model level, the cost is moderate and well understood. At the data level, the gap between synthetic and real images is large. This gap cannot be attributed to differential privacy alone. The dataset was produced at one calibration budget, and there was no non-private dataset to compare against. Differences in rendering, object size and image shape add to the gap.

Integration also caught a fault that no single component would have found alone. The composition check found that the earlier calibration release had run each query at $\varepsilon = 1.8$. The seven queries therefore spent 12.6, while the file reported 1.8. Each query's noise had the correct size, so a per-query check could not see the fault. Privacy Integration changed its module to split a total budget over the queries. The same check then passed on the corrected release. Figure 6.11 shows both results. The dataset was then captured again from the corrected release. All results in this report come from that capture.

A later review of the calibration code found one more weakness. The calibration noise was drawn from a pseudo-random generator with a fixed seed of 42. This seed was stored in a public configuration file. With the seed, the noise of each query could be generated again and removed from the published values. This would have revealed the source statistics. The reconstruction check assumes that the attacker does not know the noise, so no check detected this. Privacy Integration corrected the fault before the dataset was captured again.

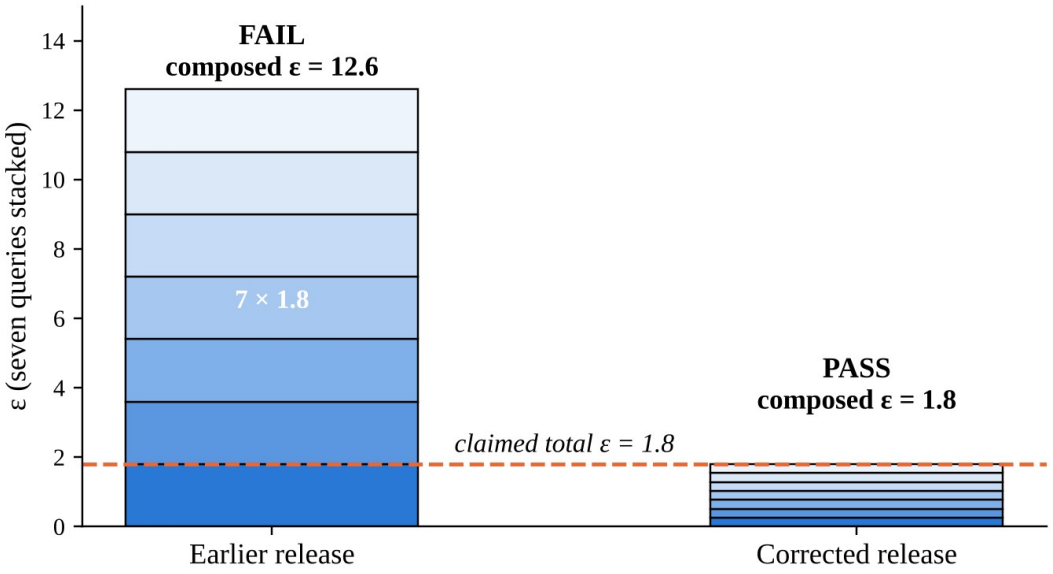


Figure 6.11: Composed calibration budget of the earlier and corrected releases

6.4 Stakeholder and user validation

Three groups reviewed the complete solution. The supervisors reviewed the work at the end of each sprint. They helped us narrow the scope to four components we could build and test. Risk and Compliance was the internal user of the evidence from the other components and verified it without their help. The progress review panels evaluated the solution at the proposal and progress stages.

At the second progress review (PP2), the panel gave one comment per component. Table 6.16 lists the comments, our response and the concern that remains.

Table 6.16: Comments of the second progress review panel and the team's responses

Component	Panel comment	Response	Remaining concern
PI	Even with differential privacy, the reduction of attacks is not significant	The drop was significant but small, because the baseline leaked only moderately. A stress test on an overfitted model showed ε = 1 removing 99.5% of the advantage	A stronger, reference-calibrated attack has been built but not yet tested
DG	How is the original file removed? Is it using crypto-shredding?	The plaintext is deleted with a normal file delete after the encrypted copy is written. This is not crypto-shredding	Encrypt in memory so plaintext is never written; per-batch keys for crypto-shredding

Component	Panel comment	Response	Remaining concern
UE	How was data poisoning detection validated?	Planted defects are caught in tests, including an out-of-range box, an unknown class and a fully relabeled batch	The batch layer also excludes natural variation, which reduces the training data
RC	Metadata leakage findings are 0	The leakage patterns are now tested with planted paths, addresses, coordinates and credentials, which the scan finds	Real calibration data would need the same checks before release

These comments led to one change across the whole solution. Each component now reports the result of its test. It also shows evidence that the test can detect a real problem. The open items are listed as future work in Section 8.3.

6.5 Achievement of common objectives

Table 6.17 maps the evidence to each group objective.

Table 6.17: Achievement of the group objectives

No.	Objective	Evidence	Status
GO1	Training budget within ε = 10	Final ε of 0.991, 3.000 and 7.995; logs equal the reports in the independent audit	Fully achieved
GO2	Attack success at or below 55%	50.3% to 51.6% with DP, 56.7% without; stress test confirms the effect	Fully achieved
GO3	Perception within 15% of real data	10.46% of the real-data mAP50 on real images	Not achieved
GO4	Calibration privatized within ε = 2	Corrected release composes to 1.8; dataset captured from it; fixed seed found and corrected	Fully achieved
GO5	Every generated file protected	SHA-256, AES-256 and RBAC; all audit tests passed; 40,000 of 40,000 files matched	Fully achieved
GO6	Labels checked for poisoning	Planted defects caught; full dataset screened (FILTERED)	Fully achieved
GO7	Independent verification and GDPR mapping	All checks passed; 8 of 8 provisions verified; signed certificate	Fully achieved

GO3 was not achieved. The detector trained on synthetic data worked inside the synthetic domain. On real images, it lost most of its performance. GO4 was achieved after two corrections. The budget fault and the fixed seed were both fixed before the dataset was captured again. Each fully achieved objective has evidence from more than one source. GO1 was checked by the independent budget audit, and GO2 by two attacks and a stress test. GO4 rests on the composition and reconstruction checks. GO5 rests on the security audit and the manifest gate. GO6 rests on planted-defect tests and the full screening. GO7 has the compliance report and the signed certificate behind it.

6.6 Integrated limitations and threats

Dependencies between components. The delivered dataset uses AES-256-CBC, which has no built-in tamper detection. Its integrity depends on the SHA-256 manifests, which the manifest gate checks. The GCM engine was tested on a separate Town01 capture. The dataset was captured at one calibration budget only. We therefore could not measure how much of the utility gap comes from calibration noise.

Validity threats. The calibration statistics came from a single source, the Waymo Open Dataset. Three seeds separated private from non-private models but not the budgets, which only the five-seed stress test separated. The attacks were realistic but not the strongest known [6]. CARLA images are 800 × 600 pixels, while KITTI images are about 1242 × 375, which affects the utility gap. The detectors were trained before the batch layer of the poisoning gate was added.

Scope of the checks. A passing statistical test shows only that no weakness was found in that property. It does not replace formal cryptanalysis. The independent checks cover only the listed evidence files. They do not cover the whole software of the other components. The fixed calibration seed is an example of a fault that every check passed. Some GDPR mappings rest on indirect evidence. For example, Article 17 is linked to k-anonymity, which shows that metadata cannot single out a frame.

Unresolved integration risks. The keys are kept in local files, without a key management service. The HMAC key is shared between the producer and the auditor, so either party could create a valid signature. The RBAC gateway has no user

authentication and keeps no persistent access log. Opacus secure mode was turned off, because the library it needs does not support current PyTorch versions.

Generalizability. The results come from one simulator, five towns, one privacy classifier and one detector family. They may not hold for other simulators, larger models or other sensor data.

7 COMMERCIALISATION SUSTAINABILITY AND RESPONSIBLE INNOVATION

7.1 Value proposition and beneficiaries

The framework could be offered as a privacy and security layer for teams that build perception models with simulated data. Such a team would receive synthetic data together with a record that another party can check. The record shows where privacy was applied and how the files were protected. It also shows what privacy cost and which independent checks passed. With it, a team can use real driving statistics in its simulation. It can still show that the statistics, files and models were protected.

The beneficiaries are the stakeholders in Table 1.1. Developers receive data they can use without a long privacy review for every dataset. Road users are less likely to have their faces, plates or routes exposed. Data protection officers and regulators receive evidence they can read and verify. Researchers receive datasets they can share with a clear record. Table 7.1 sets out the value for each group in more detail.

Table 7.1: Value of the framework for each customer or user group

Group	Value they receive
Autonomous vehicle developers	Synthetic data built from privatized statistics, with signed privacy and compliance certificates for their data protection officer
Data protection officers and regulators	One-page certificates that state the verdict, each check, the GDPR provisions verified and the known weaknesses
University research labs	Encrypted datasets with manifests that partners can check on arrival
Simulation service providers	A way to accept calibration data from clients without receiving raw recordings
Road users and the public	Less exposure of faces, plates and routes in training pipelines and trained models

Many privacy claims in this area are hard to check from outside. A framework that signs its evidence and has it checked independently makes it easier for others to question a claim. This can increase trust in synthetic data more generally.

7.2 Market alternatives and differentiation

We target autonomous vehicle start-ups, automotive suppliers, university research labs and simulation service providers. Many of them want real statistics to make their simulations realistic. However, privacy rules and contracts stop them from sharing raw recordings.

Each existing alternative covers only part of this need. Face and plate blurring removes visible identifiers but does not protect the trained model. Simulators such as CARLA and NVIDIA DRIVE Sim produce realistic data. Our proposal found no formal privacy guarantee for their calibration step. Private generative models such as DP-GAN were tested only on small benchmarks [17]. Training libraries such as Opacus [15] apply DP-SGD but do not test the result. Auditing tools such as ML Privacy Meter [38] measure leakage but neither apply privacy nor protect files.

Our framework joins these steps into one pipeline. It privatizes the calibration channel and protects every file from capture. It also measures utility against real data and has every claim checked by separate code. Table 7.2 compares the alternatives with the framework.

Table 7.2: Existing alternatives compared with the framework

Alternative	What it covers	What it leaves out
Face and plate blurring	Removes visible identifiers from real images	Trained models, calibration data and file protection
Simulators such as CARLA and NVIDIA DRIVE Sim	Realistic synthetic scenes with automatic labels	A formal privacy guarantee for the calibration step
Private generative models such as DP-GAN	Formal privacy during generation	Tests on driving data; file protection; independent checks
DP training libraries such as Opacus	DP-SGD and budget accounting	Testing whether the budget stops attacks
Auditing tools such as ML Privacy Meter	Measuring the leakage of a trained model	Applying privacy and protecting files
Our framework	Calibration privacy, file protection, utility measurement and independent checks in one pipeline	Closing the synthetic-to-real gap; production key management

Each part uses known methods on its own. Our advantage is in how the parts are joined. The contracts, the signed evidence and the independent checks were built to work together. Every control also comes with a measured cost. Rebuilding this combination would take another team considerable time.

7.3 Business model and feasibility

The most realistic business model is an open-source core with paid services. Releasing the core as an open-source toolkit lowers the barrier for research labs and builds trust in the method. Open code also matters for the checks, because users must be able to see what each one does. Companies could pay for integration with their own simulators and pipelines, and for support and training. A separate party could also run the independent checks as an audit service. Universities could use the toolkit under academic licenses.

The framework is feasible on modest hardware. Privacy training and attacks ran on a laptop GPU with 4 GB of memory. Detector training ran on a CPU, and the independent checks run in seconds. The main libraries are free and open source. The main costs would be staff time to adapt the framework to each customer's data. Larger detectors would also need GPU time. Table 7.3 summarizes the main parts of the business model. These parts are plans and have not been tested.

Table 7.3: Main parts of the business model

Element	Plan
Customer segments	Autonomous vehicle start-ups, automotive suppliers, university research labs and simulation service providers
Channels	Open-source release of the core, the project website and the research paper
Key partners	Open-source communities of the libraries we use, universities and automotive data teams
Key resources	The code of the four components, the contracts, the test suites and the team's knowledge of the pipeline
Cost structure	Staff time for integration and support, GPU time for larger models and ongoing maintenance
Revenue or sustainability	Paid integration, support and training; an independent audit service; academic licenses

Our own results give some evidence of feasibility. The whole pipeline ran on student hardware. Encrypting a 4 KB file took about 15 ms, and each GCM file grows by only 32 bytes. The file-based design worked across four members' machines. It handled real problems, such as wrong labels and an understated budget, without a redesign.

7.4 Adoption intellectual property and compliance

Adoption barriers. The utility gap is the largest barrier. A detector trained on our synthetic data kept only 10.46% of the real-data mAP50. Developers would still need real data for final training. Before production use, the framework would also need key management, user authentication and an access log.

Intellectual property. The framework uses CARLA, PyTorch, Opacus, Ultralytics, PyCryptodome and other open-source libraries. Any release must follow their licenses and keep their notices. The calibration statistics came from the Waymo Open Dataset, and KITTI was used only as a utility baseline. Both datasets are licensed for non-commercial use only. A commercial product could not use them under these licenses. The rules of the university decide who owns the code written by the group.

Data protection and ethics. All images were generated in CARLA, so no real person, vehicle or place was recorded by the project. The calibration statistics, however, came from the Waymo Open Dataset, which was recorded on public roads. Files that could reveal source values, such as the calibration audit log, went only to Risk and Compliance. The framework does not claim legal certification. Both certificates state that they cover only the listed checks. The framework helps a team prepare evidence for duties such as GDPR Articles 5, 25 and 32 [3]. The final legal decision stays with the data controller.

Safety. The framework produces training data and evidence. It is not a driving system. We did not test any model in a vehicle. A detector trained on our synthetic data is a research result. It should not be used for safety decisions without further training and validation on real data.

Professional responsibilities. As cybersecurity students, we tried to follow the duties we expect from others. We reported results that missed their target, such as GO3. We reported the calibration budget fault and the fixed seed in full. Keys were removed

from the repository once a review found them. We cited every library and dataset we used and complied with their licenses.

7.5 Sustainable Development Goal contribution

As Section 1.3 described, the likely contribution is to SDG 9 (target 9.5) and SDG 16 (target 16.10). The framework gives the automotive sector a tested way to use synthetic data while protecting the privacy of road users. Its signed evidence also supports the oversight work of regulators, which relates to accountable institutions under target 16.6.

The framework could also have negative effects. A PASS verdict could give a false sense of security if users read it without its limits. The fixed calibration seed is an example of a fault that every check passed. Training many models also uses energy. To reduce these risks, the declared weaknesses and scope notes should always travel with the certificates. A small model should be used where it is enough. A contribution to road safety under SDG 3 and SDG 11 would need the utility gap to be closed first.

Some measures would help keep these benefits over time. The checks should be updated as stronger attacks appear and run again whenever the pipeline changes. A future version should log the energy used by each training run. The cost could then be weighed against the privacy benefit.

8 CONCLUSION AND FUTURE DIRECTIONS

8.1 Overall conclusion

The common problem had four parts. The pipeline had to apply formal privacy where real information enters and protect its files. It also had to measure its utility and give evidence that others can verify. The integrated evidence answers each part.

Privacy was applied at both entry points. The calibration statistics were privatized before they reached the simulator. The corrected release stays within a total of $\varepsilon = 1.8$, and the dataset was captured from it. DP-SGD brought membership inference from an AUC of 0.639 to near chance on every private model. The cost was 0.16 to 0.21 in test AUC. On a model that memorized its training frames, $\varepsilon = 1$ removed 99.5% of the attacker's advantage. The files were protected from the moment of capture. The security audit showed that they could not be read or changed without detection. Utility was measured against real data. The synthetic data trained a working detector in its own domain. On KITTI, it kept only 10.46% of the real-data mAP50. Every privacy claim was checked by separately written code, and the final compliance report passed.

For privacy, protection and verification, the answer is therefore yes, within the limits stated in Section 6.6. For utility, the answer is not yet. The gap between synthetic and real data is the main problem that remains.

For a developer, this has a practical meaning. The framework can already bring real statistics into a simulator with a formal guarantee and protect the generated files. It produces evidence that a data protection officer can read and an auditor can test. However, the synthetic data cannot yet replace real data for the final training of a detector.

8.2 Principal integrated contribution

The main contribution of our team is a working pipeline that applies, protects, measures and verifies privacy in one flow. The integration produced four outcomes that no single component could have produced alone.

- **Evidence that crosses boundaries.** Signed reports, budget logs and manifests let one component check another without trusting it. This is how the understated calibration budget was found.
- **A shared definition of done.** Every claim needs a number, a test result and a signature. After the panel review, every check also needs evidence that it can detect a real problem.
- **Two levels of privacy cost.** Privacy Integration measured the cost at the model level, and Utility Evaluation measured it at the data level. Together they show where privacy costs the most.
- **Contracts that make the pipeline modular.** File contracts let four components be built on different machines and joined without changing their internal code.

8.2.1 Lessons learned

The project taught us lessons that we would apply to any similar system.

- **Check the data first.** When our first private models failed, DP-SGD seemed the likely cause. The actual cause was label noise and stretched images.
- **Always attack an unprotected model too.** A result near 50% is meaningless unless the same attack succeeds without protection. When the baseline leaked little, a stress test was needed to show the effect clearly.
- **Add up every budget.** Seven correctly noised queries spent seven times the reported budget. Only a check of the total found it.
- **Treat seeds as secrets.** A fixed seed makes a release repeatable, but it also lets an attacker repeat the noise. Release mechanisms need a secure random source.
- **Agree on contracts early.** Most integration problems came from details missing from a contract, such as file names, field meanings and column names.
- **Report per class and per level.** Overall scores hid the near-zero results for pedestrians and cyclists on real images.

8.3 Future directions

The future work is listed in order of priority.

1. **Seed check.** Add a check that fails on a fixed or public seed. A fault like the one found here would then be caught automatically.
2. **Close the utility gap.** Capture at several calibration budgets to draw a full privacy and utility curve. Retrain the detectors on the filtered labels and use a real baseline with a closer image shape. Then fine-tune on a small real subset.
3. **Stronger attacks and more seeds.** Test the reference-calibrated attack that Privacy Integration has built. Repeat the membership inference study with separate code in Risk and Compliance.
4. **Strengthen file protection.** Move the delivered dataset to AES-256-GCM. Encrypt frames in memory so that plaintext is never written. Keep keys in a key management service, with per-batch keys.
5. **Harden access and evidence.** Add user authentication and a tamper-evident access log to the RBAC gateway. Replace HMAC with public-key signatures, so that only the signer can sign and any party can verify.

Table 8.1 links each direction to the components involved and the objective it would strengthen.

Table 8.1: Future directions mapped to components and group objectives

Priority	Future direction	Components	Objective strengthened
1	Seed check	PI, RC	GO4, GO7
2	Close the utility gap	DG, UE	GO3
3	Stronger attacks and more seeds	PI, RC	GO2, GO7
4	Strengthen file protection	DG	GO5
5	Harden access and evidence	DG, PI, RC	GO5, GO7

The first item makes the calibration checks more complete. The second would answer the one part of the common problem that is still open. The last three move the framework from a research pipeline toward a system that others could deploy.

REFERENCES

[1] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? The KITTI vision benchmark suite," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2012, pp. 3354-3361.

[2] P. Sun *et al.*, "Scalability in perception for autonomous driving: Waymo Open Dataset," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 2446-2454.

[3] European Parliament and Council of the European Union, "Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation)," *Off. J. Eur. Union*, vol. L 119, pp. 1-88, May 2016.

[4] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in *Proc. 1st Annu. Conf. Robot Learn. (CoRL)*, ser. Proc. Mach. Learn. Res., vol. 78, 2017, pp. 1-16.

[5] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2017, pp. 3-18.

[6] N. Carlini, S. Chien, M. Nasr, S. Song, A. Terzis, and F. Tramèr, "Membership inference attacks from first principles," in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2022, pp. 1897-1914.

[7] T. Stadler, B. Oprisanu, and C. Troncoso, "Synthetic data: Anonymisation groundhog day," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 1451-1468.

[8] C. Gao, G. Wang, W. Shi, Z. Wang, and Y. Chen, "Autonomous driving security: State of the art and challenges," *IEEE Internet Things J.*, vol. 9, no. 10, pp. 7572-7595, 2022.

[9] Y. Xu, J. Wei, T. Mi, and Z. Chen, "Data security in autonomous driving: Multifaceted challenges of technology, law, and social ethics," *World Electr. Veh. J.*, vol. 16, no. 1, Art. no. 6, 2025.

[10] C. Dwork, "Differential privacy," in *Proc. 33rd Int. Colloq. Automata, Lang. Program. (ICALP)*, ser. Lecture Notes Comput. Sci., vol. 4052. Berlin, Germany: Springer, 2006, pp. 1-12.

[11] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," *Found. Trends Theor. Comput. Sci.*, vol. 9, no. 3-4, pp. 211-407, 2014.

[12] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating noise to sensitivity in private data analysis," in *Proc. 3rd Theory Cryptogr. Conf.*

(TCC), ser. Lecture Notes Comput. Sci., vol. 3876. Berlin, Germany: Springer, 2006, pp. 265-284.

[13] M. Abadi *et al.*, "Deep learning with differential privacy," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2016, pp. 308-318.

[14] I. Mironov, "Rényi differential privacy," in *Proc. IEEE 30th Comput. Secur. Found. Symp. (CSF)*, 2017, pp. 263-275.

[15] A. Yousefpour *et al.*, "Opacus: User-friendly differential privacy library in PyTorch," 2021, *arXiv:2109.12298*.

[16] Y. Wu and K. He, "Group normalization," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 3-19.

[17] L. Xie, K. Lin, S. Wang, F. Wang, and J. Zhou, "Differentially private generative adversarial network," 2018, *arXiv:1802.06739*.

[18] R. Torkzadehmahani, P. Kairouz, and B. Paten, "DP-CGAN: Differentially private synthetic data and label generation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, 2019, pp. 98-104.

[19] Y. Tian, Y. Shi, Y. Zhang, and Q. Tian, "Personalized federated learning scheme for autonomous driving based on correlated differential privacy," *Sensors*, vol. 25, no. 1, Art. no. 178, 2025.

[20] C. Xie, Z. Cao, Y. Long, D. Yang, D. Zhao, and B. Li, "Privacy of autonomous vehicles: Risks, protection methods, and future directions," 2022, *arXiv:2209.04022*.

[21] S. Yeom, I. Giacomelli, M. Fredrikson, and S. Jha, "Privacy risk in machine learning: Analyzing the connection to overfitting," in *Proc. IEEE 31st Comput. Secur. Found. Symp. (CSF)*, 2018, pp. 268-282.

[22] B. Jayaraman and D. Evans, "Evaluating differentially private machine learning in practice," in *Proc. 28th USENIX Secur. Symp.*, 2019, pp. 1895-1912.

[23] M. Nasr, S. Song, A. Thakurta, N. Papernot, and N. Carlini, "Adversary instantiation: Lower bounds for differentially private machine learning," in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2021, pp. 866-882.

[24] T. Sun *et al.*, "SHIFT: A synthetic driving dataset for continuous multi-task domain adaptation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 21371-21382.

[25] H. Caesar *et al.*, "nuScenes: A multimodal dataset for autonomous driving," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 11621-11631.

[26] A. Giannaros *et al.*, "Autonomous vehicles: Sophisticated attacks, safety issues, challenges, open topics, blockchain, and future directions," *J. Cybersecur. Privacy*, vol. 3, no. 3, pp. 493-543, 2023.

[27] *Secure Hash Standard (SHS)*, FIPS PUB 180-4, Nat. Inst. Standards Technol., Gaithersburg, MD, USA, 2015.

[28] *Advanced Encryption Standard (AES)*, FIPS PUB 197, Nat. Inst. Standards Technol., Gaithersburg, MD, USA, 2001, updated 2023.

[29] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC," Nat. Inst. Standards Technol., Gaithersburg, MD, USA, NIST Spec. Publ. 800-38D, 2007.

[30] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 779-788.

[31] G. Jocher, A. Chaurasia, and J. Qiu, *Ultralytics YOLOv8*. (2023). Ultralytics. Accessed: Oct. 10, 2026. [Online]. Available: https://github.com/ultralytics/ultralytics

[32] T.-Y. Lin *et al.*, "Microsoft COCO: Common objects in context," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, ser. Lecture Notes Comput. Sci., vol. 8693. Cham, Switzerland: Springer, 2014, pp. 740-755.

[33] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 2980-2988.

[34] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "GANs trained by a two time-scale update rule converge to a local Nash equilibrium," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 6626-6637.

[35] Z. Wang, J. Ma, X. Wang, J. Hu, Z. Qin, and K. Ren, "Threats to training: A survey of poisoning attacks and defenses on machine learning systems," *ACM Comput. Surv.*, vol. 55, no. 7, pp. 1-36, 2022.

[36] J. Hayes, L. Melis, G. Danezis, and E. De Cristofaro, "LOGAN: Membership inference attacks against generative models," *Proc. Privacy Enhancing Technol.*, vol. 2019, no. 1, pp. 133-152, 2019.

[37] B. van Breugel, H. Sun, Z. Qian, and M. van der Schaar, "Membership inference attacks against synthetic data through overfitting detection," in *Proc. 26th Int. Conf. Artif. Intell. Statist. (AISTATS)*, ser. Proc. Mach. Learn. Res., vol. 206, 2023.

[38] S. K. Murakonda and R. Shokri, "ML Privacy Meter: Aiding regulatory compliance by quantifying the privacy risks of machine learning," 2020, *arXiv:2007.09339*.

[39] L. Sweeney, "k-anonymity: A model for protecting privacy," *Int. J. Uncertainty, Fuzziness Knowl.-Based Syst.*, vol. 10, no. 5, pp. 557-570, 2002.

[40] A. Pyrgelis, C. Troncoso, and E. De Cristofaro, "Knock knock, who's there? Membership inference on aggregate location data," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2018.

[41] O. Adeboye, T. Dargahi, and M. Babaie, "DeepClean: A robust deep learning technique for autonomous vehicle camera data privacy," *IEEE Access*, vol. 10, 2022, doi: 10.1109/ACCESS.2022.3222834.

[42] *Information Security, Cybersecurity and Privacy Protection: Privacy Enhancing Data De-identification Framework*, ISO/IEC 27559:2022, Int. Org. Standardization, Geneva, Switzerland, 2022.

[43] G. Ganev, "When synthetic data met regulation," 2023, *arXiv:2307.00359*.

[44] Nat. Inst. Standards Technol., "NIST Privacy Framework: A tool for improving privacy through enterprise risk management, Version 1.0," U.S. Dept. Commerce, Gaithersburg, MD, USA, 2020.

[45] Nat. Inst. Standards Technol., "Artificial Intelligence Risk Management Framework (AI RMF 1.0)," U.S. Dept. Commerce, Gaithersburg, MD, USA, Rep. NIST AI 100-1, 2023.

[46] PyCryptodome Project, "PyCryptodome documentation," 2024. Accessed: Oct. 10, 2026. [Online]. Available: https://www.pycryptodome.org

[47] ReportLab, "ReportLab PDF library user guide," 2024. Accessed: Oct. 10, 2026. [Online]. Available: https://www.reportlab.com/docs/reportlab-userguide.pdf

APPENDICES

Appendix A Integrated interface specification

This appendix describes the files that connect the four components. Each file in Table A.1 is checked against a contract or a fixed format before it is used.

Table A.1: Integrated interface specification

Artifact	Producer	Used by	Format	Validation rule
calibration_stats.json	PI	DG, RC	JSON: seven privatized statistics and the total ε spent	Schema check; composition check
calibration_audit_log.json	PI	RC only	JSON: sensitivity, noise scale, ε spent and noise for each query	Never released; reconciliation and reconstruction checks
Encrypted image files	DG	UE, PI, RC	16-byte IV followed by AES-256-CBC ciphertext with PKCS7 padding	Decrypts with the dataset key
Town manifests	DG	UE, PI, RC	SHA-256 digest of each original file	Manifest gate: every decrypted file must match
Frame metadata	DG	RC	CSV: one row per frame with weather, time, event and town	k-anonymity check
Label files	DG	UE	YOLO format: class, normalized x, y, width, height	Three-layer poisoning gate
clean_labels/	UE	PI	Label files that passed the gate	Rejected frames excluded
audit_report.json and audit_certificate.json	PI	RC	Report with ε, C, σ, δ, limit and steps; HMAC-SHA256 tag and SHA-256 digest	Tag and digest must match
budget_log.csv	PI	RC	Cumulative ε after every training step	Equal to the report; never decreasing; at most 10

Artifact	Producer	Used by	Format	Validation rule
summary.json	PI	UE	Attack AUC and test AUC per configuration	Schema check; seeds gate
poisoning_audit.json and utility_results.json	UE	PI, RC	Gate verdict; mAP, retention and FID	Schema check
compliance_report.json	RC	Release folder	Check results, GDPR mapping, weaknesses and verdict	Schema check; certificate fingerprint
pipeline_report.json	Pipeline	Release folder	Integrated verdict of the run	Schema check

Appendix B Team contribution declaration

All four members agreed on the contributions in Table B.1. Each member owned one component, plus a share of the integration work described in Section 4.5.

Table B.1: Team contribution declaration

Student ID	Student name	Individual component	Contribution
IT22309556	Priyadarshi S.A.S.D.	Privacy Integration	25%
IT22541284	Yubitha S.	Dataset Generation	25%
IT22110220	Apeksha M.K.S.R.	Utility Evaluation	25%
IT22066916	Wijesinghe R.D.P.T.	Risk and Compliance	25%

Confirmed by all four members:

Student name	Student ID	Signature
Priyadarshi S.A.S.D.	IT22309556	
Yubitha S.	IT22541284	
Apeksha M.K.S.R.	IT22110220	
Wijesinghe R.D.P.T.	IT22066916	

Appendix C Academic integrity and AI use disclosure

We used generative AI while preparing this report, following the module policy on AI use.

Tool. Claude, a large language model by Anthropic.

Use. The tool drew the diagrams and charts from our results. It was also used for a final verification of the report before submission.

Verification. We checked the numbers, citations and claims against the individual reports, the experiment outputs and the cited sources.

Responsibility. The research design, the experiments, the results and the conclusions are the responsibility of the team.